SIMD Image Processing Library

CPU optimization: scalar, OpenMP, AVX2

The complete guide

what it is, why it is fast, how the code does it,
how to run it on bwUniCluster, and how to talk about it

Written for: Syed Akram Ameer Abbas
Project: "SIMD Image Processing Library – CPU Optimization" (C++, AVX2, OpenMP, Python), project 1 on the resume
Level of the implementation: what a solid M.Sc. graduate in an embedded / computing programme would build: readable, tested, measured, and honest about its limits
Target: bwUniCluster 3.0 (KIT), GNU toolchain, Slurm
Code: the `simd_imgproc/` directory delivered together with this PDF

Everything printed in this guide as a number, table, figure or listing was produced by the code in the project: no value was typed in by hand. Where a number is *yours to measure* (the cluster benchmarks), the guide says so.

Contents

1 The big picture **6**

1.1 What you built, in three sentences . . . 6

1.2 The three operations and the four ways of running them . . . 6

1.3 What this project is and is not . . . 6

1.4 Where everything lives . . . 7

1.5 How to use this guide . . . 7

2 Step 0: what an image is, to a computer **9**

2.1 Pixels are bytes . . . 9

2.2 How the grid is laid out in memory . . . 9

2.3 The Image class . . . 9

2.4 What happens at the edges? . . . 10

3 The three operations: maths in plain language **12**

3.1 RGB to gray . . . 12

3.1.1 The idea . . . 12

3.1.2 Doing it with integers only . . . 12

3.2 2D convolution . . . 12

3.2.1 The idea . . . 12

3.2.2 Kernels you can try . . . 13

3.3 Sobel edge detection . . . 14

3.3.1 The idea . . . 14

3.3.2 How big can the numbers get? (this decides the data type) . . . 15

4 The scalar versions, and what the compiler does by itself **16**

4.1 The structure shared by all versions . . . 16

4.2 Three levels of “slow”: reference, scalar, autovec . . . 16

4.3 The scalar code . . . 17

4.4 What did the auto-vectorizer do? . . . 18

5 Using all the cores: OpenMP **20**

5.1 The idea . . . 20

5.2 Our scheme: one contiguous band of rows per thread . . . 20

5.3 What limits the speedup? . . . 21

5.4 Controlling the threads from outside . . . 21

6 SIMD from zero: AVX2 in one chapter 23

6.1 The idea . . . 23
6.2 Reading an intrinsic's name . . . 23
6.3 Every instruction this project uses . . . 24
6.4 Two things that trip everyone up . . . 24
6.4.1 Lanes: the 256-bit register is really two 128-bit halves . . . 24
6.4.2 Tails: the image width is rarely a multiple of the vector width . . . 24
6.5 Compiling and calling it safely . . . 24

7 AVX2 grayscale, line by line 26

7.1 Problem 1: the three channels are interleaved . . . 26
7.2 Problem 2: the arithmetic must not overflow . . . 27
7.3 Problem 3: narrowing back to bytes . . . 27
7.4 The complete kernel . . . 27
7.5 Why this is only a few times faster, not 16 times . . . 28

8 AVX2 Sobel, line by line 29

8.1 Vectorizing across pixels . . . 29
8.2 The arithmetic in 16-bit lanes . . . 29
8.3 Clamping and packing for free . . . 29
8.4 The complete kernel . . . 30

9 AVX2 convolution, line by line 32

9.1 Idea 1: convert each row to float once, with the border built in . . . 32
9.2 Idea 2: a ring of k float rows, each converted only once . . . 32
9.3 Idea 3: eight accumulators in registers, one broadcast weight . . . 33
9.4 Idea 4: the same order of operations as the scalar code . . . 33
9.5 Converting back to bytes . . . 33
9.6 The complete kernel . . . 34

10 Memory, caches and the roofline 36

10.1 The memory hierarchy in one picture . . . 36
10.2 Working sets of the three operations . . . 36
10.3 Arithmetic intensity and the roofline . . . 37
10.3.1 A measured memory yardstick . . . 37
10.3.2 Compute side: how close is convolution to the FMA roof? . . . 38

10.4 Image size and the cache . . . 38
10.5 Does the row stride matter? . . . 39
10.6 Other effects the code does not exploit (and you can mention) . . . 40

11 Numerics: what "the same result" means 42
11.1 Integer operations: exact . . . 42
11.2 Floating point in one page . . . 42
11.3 The convolution contract . . . 42
11.4 An error bound that is tested, not just stated . . . 43
11.5 Rounding and clamping details that cause real bugs . . . 44
11.6 OpenCV and the 14-bit gray conversion . . . 44

12 Testing: how we know it is correct 45
12.1 The layers . . . 45
12.2 Why the edge sizes matter most . . . 45
12.3 The memory-safety trap: a bug the output cannot show . . . 46
12.4 Mutation testing: a measurement of the test suite . . . 46
12.5 The independent cross-check . . . 47
12.6 How to run everything . . . 47

13 Benchmarks: method, results, and how to read them 48
13.1 Method . . . 48
13.2 Main results . . . 49
13.3 Convolution: kernel size and the ablations . . . 50
13.4 Comparison with OpenCV . . . 51
13.5 Repeatability: how big is a "real" difference? . . . 51
13.6 What to say about performance in an interview . . . 52

14 Running it yourself: build, tools, experiments 53
14.1 Project layout . . . 53
14.2 Build and test . . . 53
14.3 The command-line tool . . . 53
14.4 The benchmark tool . . . 54
14.5 Python scripts . . . 54
14.6 Experiments . . . 54
14.7 Using the library from your own code . . . 54

15 Running on bwUniCluster 3.0: step by step 56

15.1 The workflow in one picture . . . 56
15.2 One-time setup: account and login . . . 56
15.3 Where to put files . . . 57
15.4 Step 1: copy the project to the cluster . . . 57
15.5 Step 2: look around . . . 57
15.6 The queues you will use . . . 57
15.7 Step 3: build and test on a compute node . . . 58
15.8 Step 4: the benchmark jobs . . . 58
15.8.1 What the #SBATCH lines mean . . . 59
15.8.2 Benchmark hygiene (what makes a number trustworthy) . . . 59
15.9 Step 5: profile . . . 59
15.10 Step 6: bring the results home . . . 60
15.11 When something goes wrong . . . 60
15.12 How to describe it truthfully . . . 60
15.13 Sources . . . 61

16 The resume bullets, one by one 62
16.1 Bullet 1: three operations, three implementations . . . 62
16.2 Bullet 2: vectorization and cache-aware access . . . 62
16.3 Bullet 3: benchmarking across resolutions and kernel sizes . . . 63
16.4 Bullet 4: correctness testing . . . 63
16.5 Bullet 5: Python tooling . . . 63
16.6 Wording to consider once your cluster numbers exist . . . 64

17 Interview questions and answers 65
17.1 Orientation . . . 65
17.2 SIMD and AVX2 . . . 65
17.3 Convolution . . . 66
17.4 OpenMP and memory . . . 67
17.5 Numerics and testing . . . 68
17.6 Benchmarks . . . 68
17.7 Connecting to GPU-oriented image and data-processing libraries . . . 69
17.8 Questions to ask them . . . 69

18 Drills and an eight-day plan 71
18.1 Drills with short answers . . . 71
18.2 Code-reading drills . . . 71

18.3 Whiteboard exercises 72
18.4 Eight-day plan 72

A Glossary **74**

Chapter 1

The big picture

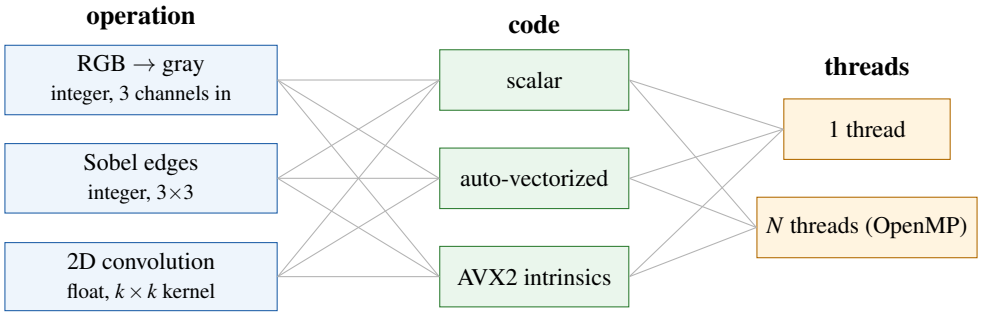
1.1 What you built, in three sentences

You built a small C++ library that performs three classic image operations on 8-bit pictures: **colour to gray conversion**, **Sobel edge detection** and **2D convolution** (blur, sharpen, … with kernels of any odd size). Each operation exists in several versions that compute *exactly the same thing* but use the processor differently: plain scalar code, the same code with the compiler's auto-vectorizer switched on, hand-written **AVX2** instructions that handle 8 to 32 numbers at once, and each of these split across several CPU cores with **OpenMP**. Then you measured how much faster each version is, checked that it is still correct, and looked for the reasons.

In plain words

The one-sentence version for an interview: *"I implemented grayscale, Sobel and 2D convolution in scalar, OpenMP and AVX2 form, proved the fast versions equal to a simple reference, and measured speedup, throughput and where each kernel is limited by compute or by memory."*

1.2 The three operations and the four ways of running them



Every operation is available in every combination ($3 \times 3 \times 2$). The *correctness contract* is: all combinations produce the same pixels. For gray and Sobel the results are identical bit for bit. For convolution the AVX2 result is identical to a reference that uses fused multiply-adds, and the scalar result differs from it by at most one grey level in a handful of rare cases that this guide explains (Chapter 11).

1.3 What this project is and is not

Being precise about scope makes you more credible, so here it is plainly.

- **It is** a complete, tested, measured CPU-optimization study: three kernels, scalar/OpenMP/AVX2 implementations, a reference-based test suite (over 4 million checks including an exhaustive test of all 16.7 million RGB colours), a benchmark harness with a memory-bandwidth yardstick, and experiments on caches, strides, kernel sizes and threads.

- **It is not** a replacement for OpenCV, Intel IPP or NVIDIA NPP. It handles 8-bit images only, one border mode (edge replication), and square odd kernels. It does not use AVX-512, does not do tiling across multiple rows at once, and does not implement separable-kernel shortcuts. Chapter 13 compares it with OpenCV on one machine so you can speak about the gap honestly.
- **The numbers in this PDF** were measured in the sandbox where the code was written (a virtual machine with two cores). They show *what the experiments look like and how to read them*. The **bwUniCluster numbers are yours to measure**: Chapter 15 gives the exact steps and the scripts. Never quote a sandbox number as a cluster number.

Honesty rule for the interview

Say "SIMD image processing library, grayscale / Sobel / convolution, scalar vs. OpenMP vs. AVX2" exactly as the resume does. Anyone who knows SIMD will respect a precise list of what you implemented and what you deliberately left out far more than a vague claim like "a high-performance imaging library".

1.4 Where everything lives

File	What it does
`include/simdip/image.hpp`, `src/image.cpp`	`Image`: aligned rows, stride, PNM file I/O, synthetic test images.
`include/simdip/kernel.hpp`, `src/kernel.cpp`	Filter kernels (box, Gaussian, binomial, sharpen, ...) and their validation.
`include/simdip/ops.hpp`	The public API: `rgbToGray`, `sobel`, `convolve`, plus the reference and ablation functions.
`src/scalar_impl.inc`	The plain C++ code. Compiled twice: `scalar_novec.cpp` (vectorizer off) and `scalar_autovec.cpp` (vectorizer on).
`src/avx2.cpp`	Hand-written AVX2 kernels.
`src/dispatch.cpp`	Argument checks, CPU feature check, splitting rows over OpenMP threads.
`src/reference.cpp`	Slow, obviously-correct reference implementations used by the tests.
`apps/simdip_cli.cpp`, `apps/simdip_bench.cpp`	Command line tool; benchmark harness (CSV output).
`tests/test_main.cpp`	24 test groups, no framework needed.
`python/ref_numpy.py`, `crosscheck.py`	Independent NumPy/SciPy model and the C++ vs. NumPy comparison.
`python/compare_opencv.py`	Agreement and speed comparison with OpenCV.
`python/run_experiments.py`	Runs experiments E1–E5 and writes the CSVs.
`slurm/*.sbatch`	Job scripts for bwUniCluster 3.0.

1.5 How to use this guide

1. Read Chapters 2–3 slowly, with the code open next to you. They explain the pictures, the three operations and the maths in plain language.
2. Chapters 4–9 walk through the code: scalar, OpenMP, the SIMD basics, and the three AVX2 kernels.
3. Chapters 10–12 are the engineering: cache behaviour, numerical accuracy, testing. This is what turns "I wrote intrinsics" into "I know why it is fast and why it is correct".
4. Chapter 13 shows the measurements; Chapter 15 is the step-by-step bwUniCluster guide. Chapters 16–18 are the interview material.

Each chapter ends with an **Interview soundbite** you can say aloud in thirty seconds.

Chapter 2

Step 0: what an image is, to a computer

2.1 Pixels are bytes

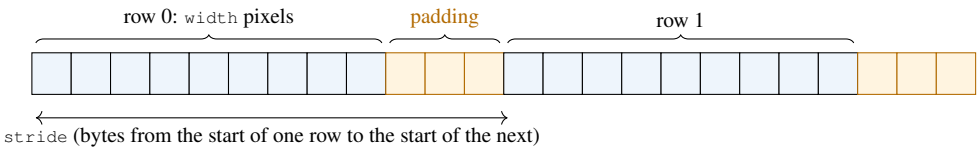
A grayscale picture is a grid of numbers from 0 (black) to 255 (white). One number fits in one *byte* (8 bits), and that byte is called a *pixel*. A colour picture has three numbers per pixel: how much red, green and blue light. A 1920×1080 gray image is about 2 MB; the same picture in colour is about 6 MB.

In plain words

Why 8 bits? It is what cameras, screens and JPEG files use, and it keeps the data small. The price is that arithmetic on pixels easily overflows: $200 + 100$ does not fit in a byte. Much of the care in this project is about doing arithmetic in wider types (16-bit integers or 32-bit floats) and converting back with *clamping* (values above 255 become 255, below 0 become 0).

2.2 How the grid is laid out in memory

Memory is a long line of bytes, so the 2D grid has to be flattened. We store it **row by row** ("row-major"), and for colour images the three channels of one pixel sit next to each other ("interleaved": `R G B R G B` ...).



The address of the pixel in column x and row y is

$$\text{address}(x, y) = \text{base} + y \cdot \text{stride} + x \cdot \text{channels}.$$

The **stride** is at least the row length, and may be larger: the extra bytes are *padding*. Real image libraries do this for two reasons, and so does this one:

1. **Alignment**: every row starts at a multiple of 64 bytes (the size of a cache line), which makes vector loads and stores efficient.
2. **Avoiding a cache trap**: if the stride is an exact multiple of 4096 bytes (a page), rows that are read together can fight for the same few cache slots. `Image::defaultStride()` adds one cache line in that case (Chapter 10 measures whether it matters on your machine).

The kernels never read or write the padding as pixels; a test fills the padding with a marker value and checks that it is unchanged afterwards (`outputs_never_touch_the_row_padding`).

2.3 The Image class

```
class Image {
public:
    Image() = default;
    // channels: 1 (gray) or 3 (RGB). stride = 0 means "use defaultStride()".
    Image(int width, int height, int channels = 1, std::size_t stride = 0);
    Image(const Image& other);              // deep copy (keeps the stride of `other`)
    Image& operator=(const Image& other);
    Image(Image&&) noexcept = default;
    Image& operator=(Image&&) noexcept = default;

    int width() const { return width_; }
    int height() const { return height_; }
    int channels() const { return channels_; }
    std::size_t stride() const { return stride_; }
    std::size_t rowBytes() const { return static_cast<std::size_t>(width_) * channels_; }
    bool empty() const { return data_ == nullptr; }

    std::uint8_t* data() { return data_.get(); }
    const std::uint8_t* data() const { return data_.get(); }
    std::uint8_t* row(int y) { return data_.get() + static_cast<std::size_t>(y) * stride_; }
    const std::uint8_t* row(int y) const { return data_.get() + static_cast<std::size_t>(y) *
     stride_; }

    // Set every byte, including the row padding.
    void fillAll(std::uint8_t v);
    // Set only the pixel bytes of each row (not the padding).
    void fillPixels(std::uint8_t v);
    // True when the pixel bytes are equal (padding is ignored). Dimensions must match.
    bool samePixels(const Image& o) const;

    static std::size_t defaultStride(int width, int channels);

private:
    int width_ = 0, height_ = 0, channels_ = 0;
    std::size_t stride_ = 0;
    std::unique_ptr<std::uint8_t, AlignedFree> data_;
};
```

Notes on what a reviewer looks for: ownership is by a `unique_ptr` with a custom deleter (no leaks, no manual `free`); copying is deep, moving is cheap; the constructor validates sizes; memory is touched (zeroed) at construction so that page faults happen *before* any timing starts; and `alignedAlloc` requests the *exact* number of bytes, so a sanitizer (AddressSanitizer) sees where the buffer really ends and reports a vector load that reads even one byte too far.

2.4 What happens at the edges?

A 3×3 filter looking at a pixel in the first column needs a neighbour that does not exist. Some choice must be made; the choice is a *border mode*. Ours is **replicate** (also called "clamp to edge"): a missing pixel takes the value of the nearest real one. This is OpenCV's `BORDER_REPLICATE`, and it never invents dark or bright values.

a	a	b	c	c
a	a	b	c	c
d	d	e	f	f
g	g	h	i	i
g	g	h	i	i

The 3×3 image (blue) with its replicated border (orange). A 3×3 filter centred on the top-left pixel `a` sees `a a b / a a b / d d e`.

Interview soundbite

"Images are stored row-major with a stride, 64-byte aligned rows, and edge-replicated borders. The stride is padded so rows are cache-line aligned and never an exact multiple of a page; the kernels never touch the padding, which a test verifies."

Chapter 3

The three operations: maths in plain language

3.1 RGB to gray

3.1.1 The idea

Gray is a weighted average of red, green and blue. The weights are not equal because the human eye is most sensitive to green and least to blue. The standard (ITU-R BT.601) weights are

$$Y = 0.299\,R + 0.587\,G + 0.114\,B.$$

A pure green pixel therefore looks much brighter in gray than a pure blue one of the same intensity.

3.1.2 Doing it with integers only

Floating-point is not needed, and integer arithmetic is easier to make exact and fast. Multiply the three weights by 256 and round: $0.299 \cdot 256 = 76.5 \to 77$, $0.587 \cdot 256 = 150.3 \to 150$, $0.114 \cdot 256 = 29.2 \to 29$. These add up to exactly 256, so a white pixel stays white ($255 \cdot 256/256 = 255$). Dividing by 256 is the same as shifting right by 8 bits:

$$Y = \left\lfloor \frac{77R + 150G + 29B + 128}{256} \right\rfloor = (77R + 150G + 29B + 128) \gg 8. \tag{3.1}$$

The "+128" is half of 256: it turns the floor (rounding down) into rounding to the nearest integer. The coefficients are the ones JPEG-style converters use; the quick check is

$$R = 255, G = 0, B = 0 \;\Rightarrow\; (77 \cdot 255 + 128) \gg 8 = 19\,763 \gg 8 = 77, \qquad R = G = B = 255 \;\Rightarrow\; (65\,280 + 128) \gg 8 = 255.$$

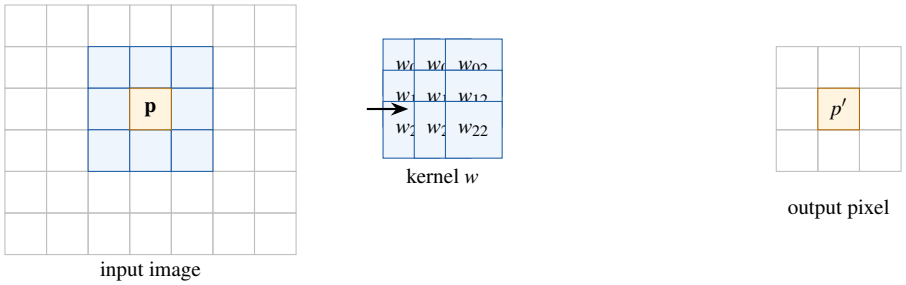
In plain words

Why this matters for SIMD. The largest possible intermediate value is $77 \cdot 255 + 150 \cdot 255 + 29 \cdot 255 + 128 = 65\,408$, which is smaller than $2^{16} = 65\,536$. So the whole computation fits in *unsigned 16-bit* numbers and a 256-bit register can hold 16 of them at once. The test `gray_exhaustive_all_16777216_colours` runs every one of the 256^3 colours through every implementation: a proof by exhaustion, not a sample.

3.2 2D convolution

3.2.1 The idea

A *kernel* is a small grid of weights, for example 3×3. To filter an image, slide the kernel over every pixel; at each position multiply the weights with the pixels underneath, add up, and write the sum to the output pixel.



In formulas, for a kernel of size $k = 2r + 1$:

$$\text{out}(x, y) = \text{clamp}\left(\text{round}\left(\sum_{j=0}^{k-1}\sum_{i=0}^{k-1} w_{ji}\,\text{in}(x+i-r,\ y+j-r)\right), 0, 255\right). \tag{3.2}$$

Each output pixel costs k^2 multiplications and k^2 additions: 3×3 is 9, 9×9 is 81, 31×31 is 961. That cost growth is why convolution is the classic target for optimization.

"Convolution" versus "correlation"

Mathematically, convolution flips the kernel before sliding it; the operation above, without the flip, is called *correlation*. Image libraries (OpenCV's `filter2D`, for example) implement the unflipped version and call it convolution. For kernels that are symmetric the two are identical; for an asymmetric kernel such as *emboss* the result is mirrored. If an interviewer asks, say which one you implemented: correlation, same as OpenCV.

3.2.2 Kernels you can try

name	kernel	effect
box (blur)	$\frac{1}{9}\begin{pmatrix}1&1&1\\1&1&1\\1&1&1\end{pmatrix}$	each pixel becomes the average of its neighbourhood: smooths noise, loses detail.
binomial	$\frac{1}{16}\begin{pmatrix}1&2&1\\2&4&2\\1&2&1\end{pmatrix}$	a blur that weights the centre more; the weights are exact binary fractions, which makes it perfect for tests (Chapter 12).
Gaussian	bell-shaped weights, σ sets width	the "natural" blur; the larger σ (and k), the stronger.
sharpen	$\begin{pmatrix}0&-1&0\\-1&5&-1\\0&-1&0\end{pmatrix}$	centre minus neighbours added back: exaggerates edges. Weights sum to 1, so overall brightness is unchanged.
identity	a single 1 in the middle	output equals input (a test).

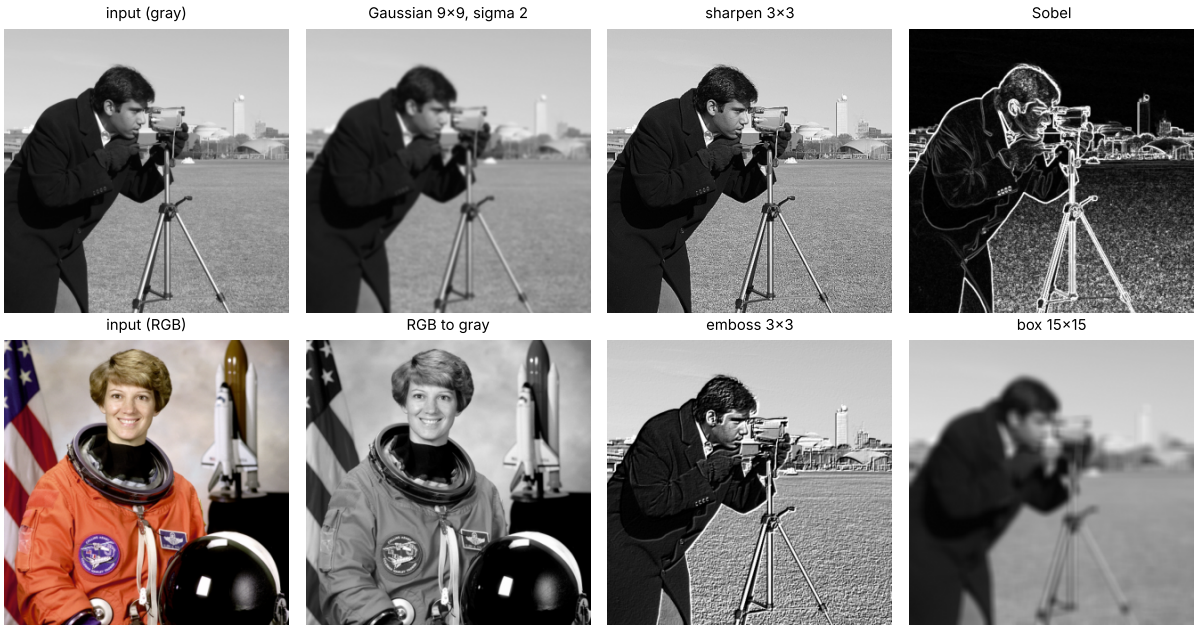


Figure 3.1: The operations on the standard "cameraman" photograph and on a colour image. Produced by the command line tool.

3.3 Sobel edge detection

3.3.1 The idea

An edge is a place where brightness changes quickly. The change of brightness in a direction is a *derivative*; for pixels it is the difference of neighbours. The simplest horizontal difference is $\mathrm{in}(x+1)-\mathrm{in}(x-1)$. Differences amplify noise, so Sobel first *smooths* across the other direction using weights $1,2,1$ and only then differences:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix} * \mathrm{in} = \underbrace{\begin{pmatrix} 1 \\ 2 \\ 1 \end{pmatrix}}_{\text{smooth vertically}} \cdot \underbrace{\begin{pmatrix} -1 & 0 & 1 \end{pmatrix}}_{\text{difference horizontally}} * \mathrm{in}, \qquad G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix} * \mathrm{in}.$$

G_x responds to vertical edges, G_y to horizontal edges. The edge strength combines both:

$$\mathrm{out} = \min\bigl(255,\ |G_x| + |G_y|\bigr) \qquad \text{(the exact length would be } \sqrt{G_x^2 + G_y^2}\text{)}. \tag{3.3}$$

We use $|G_x|+|G_y|$, the common "L1" shortcut: it needs no square root, so it stays in integers and in 16-bit lanes. It overestimates the true length for diagonal edges by at most a factor $\sqrt{2}$, which is irrelevant for a detector that is thresholded afterwards, but you should know that it is an approximation.

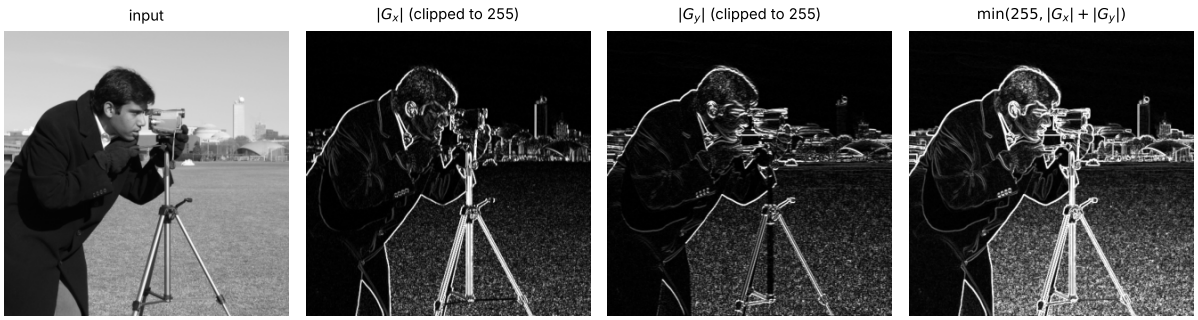


Figure 3.2: Left to right: the photograph, $|G_x|$ (vertical edges), $|G_y|$ (horizontal edges), and the final edge strength $\min(255, |G_x| + |G_y|)$.

3.3.2 How big can the numbers get? (this decides the data type)

Each of G_x, G_y is a sum of weights $\{1, 2, 1\}$ times pixels in $[0, 255]$ with the positive and negative halves separated. The worst case is $4 \cdot 255 = 1020$ in magnitude, so $|G_x| + |G_y| \leq 2040 < 32\,768$. Everything fits in **signed 16-bit** integers (16 per 256-bit register) with no overflow: no wider type is needed, so no speed is wasted. Only the last step needs care: values above 255 must be clamped when converting back to 8 bits, which the instruction `packus` does for free (Chapter 8).

Interview soundbite

"Gray uses the 77/150/29 fixed-point weights, exact in 16-bit unsigned arithmetic, and I proved it for all 16.7 million colours. Sobel is the separable $[1\,2\,1] \otimes [-1\,0\,1]$ pair with an L1 magnitude, which stays within 16-bit signed lanes. Convolution is a $k \times k$ correlation in float32 with replicated borders and round-to-nearest-even output."

Chapter 4

The scalar versions, and what the compiler does by itself

"Scalar" means one number at a time: an ordinary loop. It is the baseline against which every speedup in this project is measured, so it must be (a) correct and (b) honest: not artificially slow.

4.1 The structure shared by all versions

Every implementation exposes the same three functions that compute only the output rows $y_0 \dots y_1 - 1$:

```
void rgbToGrayRows(const Image& src, Image& dst, int y0, int y1);
void sobelRows(const Image& src, Image& dst, int y0, int y1);
void convolveRows(const Image& src, Image& dst, const Kernel& k, int y0, int y1);
```

This tiny design decision pays for itself twice: OpenMP only has to hand each thread a range of rows (Chapter 5), and the tests can call the same row functions for any slice. The public functions in `dispatch.cpp` validate the arguments (sizes, channels, no overlap of source and destination, usable kernel) and then call the right implementation.

4.2 Three levels of "slow": reference, scalar, autovec

reference	Written for obviousness. Every pixel read goes through a helper that clamps the coordinates; Sobel is computed from the two 3×3 kernels literally; convolution is a double loop in float *with an explicit fused multiply-add*, and a double-precision twin. Used only by the tests.
scalar	The code a careful programmer would write for speed: row pointers hoisted out of the loops, border pixels handled separately from the interior (so the interior has no clamping), `__restrict` on pointers. Compiled with `-O3 -fno-tree-vectorize -fno-tree-slp-vectorize`: the auto-vectorizer is **off**.
autovec	Exactly the same source file, compiled with plain `-O3`: the auto-vectorizer is **on** (but with only the baseline instruction set, SSE2, unless you build with `-march=native`).

In plain words

Why two builds of the same source? The compiler can already turn some loops into SIMD loops. If you compare hand-written AVX2 only with a loop that the compiler was forbidden to vectorize you exaggerate your speedup. Comparing with an auto-vectorized build too tells the truth: how much did the compiler give you for free, and what did the hand-written intrinsics add on top? Both are in `src/scalar_impl.inc`, #included into two translation units with different flags (`Makefile`). Both use `-ffp-contract=off` so that `a*b+c` stays a multiply followed by an add and the two builds give identical bits.

4.3 The scalar code

Three small helpers are shared by the loops: `clampInt` (limit an integer to a range, used for the replicated border), `sobelPixel` (the Sobel formula for one pixel, used for border pixels and tails by the AVX2 code too) and `roundClampF` (round-half-even and clamp to 0..255).

```
inline int clampInt(int v, int lo, int hi) { return v < lo ? lo : (v > hi ? hi : v); }

inline std::uint8_t sobelPixel(const std::uint8_t* a, const std::uint8_t* b, const std::uint8_t* c,
                               int xm, int x, int xp) {
    // a, b, c = rows y-1, y, y+1 ; xm, x, xp = columns x-1, x, x+1 (already clamped by the caller)
    const int gx = (a[xp] - a[xm]) + 2 * (b[xp] - b[xm]) + (c[xp] - c[xm]);
    const int gy = (c[xm] + 2 * c[x] + c[xp]) - (a[xm] + 2 * a[x] + a[xp]);
    const int m = std::abs(gx) + std::abs(gy);
    return static_cast<std::uint8_t>(m > 255 ? 255 : m);
}

inline std::uint8_t roundClampF(float v) {
    const long r = std::lrintf(v);   // round to nearest, ties to even (default rounding mode)
    return static_cast<std::uint8_t>(r < 0 ? 0 : (r > 255 ? 255 : r));
}
```

The row functions that use them:

```
void rgbToGrayRows(const Image& src, Image& dst, int y0, int y1) {
    const int w = src.width();
    for (int y = y0; y < y1; ++y) {
        const std::uint8_t* __restrict p = src.row(y);
        std::uint8_t* __restrict q = dst.row(y);
        for (int x = 0; x < w; ++x) {
            const unsigned r = p[3 * x], g = p[3 * x + 1], b = p[3 * x + 2];
            q[x] = static_cast<std::uint8_t>((77u * r + 150u * g + 29u * b + 128u) >> 8);
        }
    }
}

void sobelRows(const Image& src, Image& dst, int y0, int y1) {
    const int w = src.width(), h = src.height();
    for (int y = y0; y < y1; ++y) {
        const std::uint8_t* __restrict a = src.row(y > 0 ? y - 1 : 0);
        const std::uint8_t* __restrict b = src.row(y);
        const std::uint8_t* __restrict c = src.row(y < h - 1 ? y + 1 : h - 1);
        std::uint8_t* __restrict q = dst.row(y);
        if (w == 1) {
            q[0] = sobelPixel(a, b, c, 0, 0, 0);
            continue;
        }
        q[0] = sobelPixel(a, b, c, 0, 0, 1);
        for (int x = 1; x < w - 1; ++x) q[x] = sobelPixel(a, b, c, x - 1, x, x + 1);
        q[w - 1] = sobelPixel(a, b, c, w - 2, w - 1, w - 1);
    }
}
```

```
void convolveRows(const Image& src, Image& dst, const Kernel& K, int y0, int y1) {
    const int w = src.width(), h = src.height(), k = K.size, r = k / 2;
    const float* __restrict wt = K.w.data();
    const std::uint8_t* rows[kMaxKernelSize];
    for (int y = y0; y < y1; ++y) {
        for (int j = 0; j < k; ++j) rows[j] = src.row(clampInt(y - r + j, 0, h - 1));
        std::uint8_t* __restrict q = dst.row(y);
        for (int x = 0; x < w; ++x) {
            float acc = 0.0f;
```

```
            if (x >= r && x < w - r) {                        // interior: no clamping needed
                for (int j = 0; j < k; ++j) {
                    const std::uint8_t* s = rows[j] + (x - r);
                    for (int i = 0; i < k; ++i) acc += wt[j * k + i] * static_cast<float>(s[i]);
                }
            } else {                                          // left/right border: clamp the column
                for (int j = 0; j < k; ++j)
                    for (int i = 0; i < k; ++i)
                        acc += wt[j * k + i] * static_cast<float>(rows[j][clampInt(x - r + i, 0, w -
    1)]);
            }
            q[x] = roundClampF(acc);
        }
    }
}
```

Points worth mentioning in a review:

- In Sobel, `sobelPixel` gets the row pointers $y-1, y, y+1$ *already clamped* at the top and bottom of the image, and the clamped column indices. The interior loop (`x = 1 ... w-2`) passes `x-1,x,x+1` with no clamping.
- In convolution the accumulation order is fixed: `j` (kernel row) outer, `i` (kernel column) inner. **This order is a contract.** All other versions use the same order for each output pixel, which is what makes them comparable bit for bit (Chapter 11).
- `roundClampF` uses `lrintf`, which rounds to nearest, ties to even (the default CPU rounding mode), the same rule as the AVX2 instruction `cvtps2dq`.

4.4 What did the auto-vectorizer do?

The Makefile target `make vec-report` asks GCC to report every loop it vectorized in the auto-vectorized file (`-fopt-info-vec`). The result in this build:

loop	source line	vectorized by GCC?	reason if not
gray: loop over x (3 interleaved channels)	scalar_impl.inc:40	no	failed
Sobel: interior loop over x	scalar_impl.inc:59	**yes** (16-byte vectors)	
convolution: loop over x with the k^2 taps inside	scalar_impl.inc:71	no	loop nest containing two or more consecutive inner loops cannot be vec
convolution row-accumulate rewrite: `a[x] += wv * s[x]`	scalar_autovec.cpp:38	**yes** (16-byte vectors)	

In plain words

Why the natural convolution loop is not vectorized. In the natural loop the k^2 taps of one output pixel are summed in the *innermost* loops. There are two ways to use SIMD lanes on that: (a) vectorize the tap sum itself, which means adding the terms in a different order, and floating-point addition is not associative ($(a+b)+c \neq a+(b+c)$ in general), so GCC may not do it without `-ffast-math` (which also changes the results); or (b) vectorize over *neighbouring pixels* (the outer x loop), which keeps every pixel's order intact but requires vectorizing an outer loop that contains inner loops. For this loop nest the compiler's report says only "loop nest containing two or more consecutive inner loops cannot be vectorized" and fails the data-reference analysis of the inner loops; it does

not say that associativity is the reason, and it would be wrong to claim so. What we can show by experiment: a programmer can restructure the loops so that pixels become the innermost dimension; `ablation::convolveAutoVecRowAcc` does that without changing a single bit of the result, GCC then vectorizes it, and Chapter 13 measures how far this gets compared with hand-written AVX2. Integer Sobel has no ordering problem and a simple loop, which is why GCC vectorizes it.

Interview soundbite

"Scalar and autovec are the same source compiled with and without the vectorizer, so the comparison with intrinsics is fair. GCC vectorizes the integer Sobel loop but not the natural float convolution loop nest; restructuring the loops so pixels are the innermost dimension makes it vectorize, with identical result bits."

Chapter 5

Using all the cores: OpenMP

5.1 The idea

A modern CPU has many cores, each able to run its own thread. Image filters are ideal for this: the output rows are independent of each other (an output pixel depends only on the *input*, never on other output pixels), so rows can be computed in any order, on any core, at the same time. This kind of problem is called *embarrassingly parallel* or *data parallel*.

OpenMP is a standard way to tell the compiler "run this block on several threads". You write a `#pragma` and the compiler and runtime (on GNU/Linux: `libgomp`) create and manage the threads.

5.2 Our scheme: one contiguous band of rows per thread

```
// Calls f(y0, y1) for contiguous bands of rows. With threads <= 1 (or no OpenMP) it is one plain
    call.
// Band t of n covers rows [h*t/n, h*(t+1)/n): sizes differ by at most one row, each thread works
    on
// memory that is contiguous, and the same thread count always gives the same split.
template <class F>
void forBands(int h, int threads, F f) {
#ifdef _OPENMP
    if (threads > 1 && h > 1) {
#pragma omp parallel num_threads(threads)
        {
            const int t = omp_get_thread_num(), n = omp_get_num_threads();
            const int y0 = static_cast<int>(static_cast<long long>(h) * t / n);
            const int y1 = static_cast<int>(static_cast<long long>(h) * (t + 1) / n);
            if (y0 < y1) f(y0, y1);
        }
        return;
    }
#else
    (void)threads;
#endif
    f(0, h);
}
```

With n threads the image is cut into n horizontal bands of nearly equal height. Why *contiguous bands* rather than letting each thread take single rows in turn (OpenMP's default `schedule(static,1)`-like interleaving)?

- **Locality.** Each thread streams through one contiguous block of memory, and its prefetcher can follow it. Neighbouring rows (needed by Sobel and convolution) are already in that thread's cache.
- **No false sharing.** Threads write to different parts of the output; only a few cache lines at the band boundaries are shared, and only for reading the *input*.
- **Determinism.** The same thread count always gives the same split. And because each output pixel is computed from the input alone, using any number of threads gives **identical output**: the test `conv_`

`threads_do_not_change_the_result` runs 1 to 200 threads and demands identical bits.

The price: if the work per row differs a lot, equal bands are unequal work (*load imbalance*). For these kernels every row costs the same, so static bands are ideal; for irregular work you would use `schedule(dynamic)`.

In plain words

Edge cases handled on purpose. If the number of threads exceeds the number of rows (the test uses 200 threads on 97 rows) some threads receive an empty band and simply do nothing. With `threads = 1` (or an image with a single row) no OpenMP region is started at all: the function is called directly, so the single-thread benchmarks contain no threading overhead. If the library is built without OpenMP (`make OPENMP=0`) the `threads` argument is ignored.

5.3 What limits the speedup?

- **Amdahl's law.** If a fraction s of the work cannot be parallelized, the speedup on n threads is at most $1/(s+(1-s)/n)$. Here the serial parts are tiny (argument checks, thread start-up), so the limit is the next item.
- **Memory bandwidth.** All cores share the path to main memory. A kernel that does very little arithmetic per byte (gray: 3 bytes in, 1 byte out, a few operations) can saturate the memory system with only a few threads; adding more cores then adds nothing. A kernel with a lot of arithmetic per byte (convolution with large k) keeps scaling. Chapter 10 quantifies this with a measured bandwidth yardstick.
- **Thread start-up cost.** Creating or waking threads costs microseconds. For a tiny image this can exceed the work itself. (The benchmark's warm-up keeps the thread pool alive between calls.)
- **CPU frequency and shared caches.** Running many cores can lower the clock speed and makes cores share the last-level cache.

5.4 Controlling the threads from outside

`OMP_NUM_THREADS=8`	Default thread count if the program does not ask. Our API always passes an explicit count.
`OMP_PROC_BIND=close`	Pin threads to cores and keep them adjacent; stops the operating system from moving them around between cores during a measurement.
`OMP_PLACES=cores`	What a "place" is: one physical core (avoids using both hyper-threads of one core for two threads).
`taskset -c 0-7` / Slurm `-cpus-per-task`	Restrict the process to specific cores. On a Slurm node you get exactly the cores you asked for.

The benchmark prints `OMP_PROC_BIND` and `OMP_PLACES` at the top of its output so you can see how a measurement was made. The Slurm job scripts set them (Chapter 15).

First touch

On a machine with several memory controllers (NUMA, e.g. two-socket nodes), a memory page lives near the core that first wrote to it. If one thread initializes the whole image and then many threads process it, some threads read "remote" memory, which is slower. Our `Image` constructor zeroes the buffer on the calling thread. For a single-socket benchmark with pinned threads this is fine; for a serious multi-socket study you would initialize the buffers in parallel with the same band split. This is a known limitation, worth stating if asked.

Interview soundbite

"Each thread gets one contiguous band of output rows. Output pixels depend only on the input, so there is no synchronization and the result is bit-identical for every thread count, which my tests check from 1 to 200 threads. Scaling is limited by memory bandwidth for the low-arithmetic kernels, and I measured a copy bandwidth to show where the ceiling is."

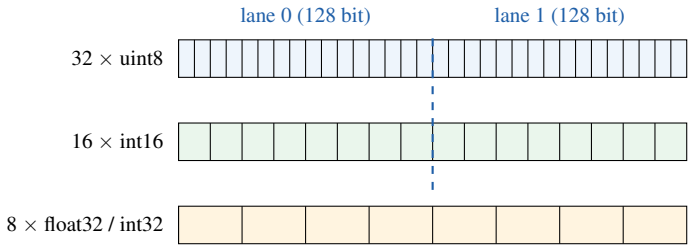
Chapter 6

SIMD from zero: AVX2 in one chapter

6.1 The idea

SIMD means *Single Instruction, Multiple Data*: one instruction operates on several numbers at once. Think of a checkout with one cashier scanning one item at a time (scalar) versus a conveyor that lets the cashier stamp 8 or 32 identical items with one press (SIMD).

Inside the CPU there are special wide registers. In **AVX2** they are **256 bits = 32 bytes** wide (named `ymm0`–`ymm15`). What is in the register is up to the instruction:



The same 256 bits can be read as 32 bytes, 16 half-words, or 8 floats. One `vaddps` adds 8 pairs of floats; one `vpaddw` adds 16 pairs of 16-bit integers. Peak throughput therefore goes up by 8 to 32 times for the same number of instructions, *if* the data is arranged so that the lanes do useful, independent work.

6.2 Reading an intrinsic's name

You write SIMD code in C++ with **intrinsics**: functions (declared in `<immintrin.h>`) that the compiler turns into one machine instruction each. The names follow a pattern:

`_mm256_add_epi16` = `_mm256` (256-bit register) `add` (operation) `epi16` (16-bit signed int elements)

Type suffixes: `epi8/16/32/64` signed integers, `epu8/16` unsigned, `ps` packed single-precision floats, `si256` "whole register, no element meaning". `_mm_` (no number) means 128-bit. The C++ types are `__m256` (8 floats), `__m256i` (integers of any width), `__m128i`.

6.3 Every instruction this project uses

intrinsic	what it does
`_mm_loadu_si128` / `_mm_storeu_si128`	load / store 16 bytes from / to *any* address ("u" = unaligned allowed).
`_mm256_loadu_ps`, `_mm256_storeu_si256`	the same for 8 floats / 32 bytes.
`_mm_shuffle_epi8` (`pshufb`)	for each output byte, pick the input byte named by an index (inside one 128-bit register); an index with the top bit set writes 0. The deinterleaving tool of Chapter 7.
`_mm256_cvtepu8_epi16`	*widen*: 16 bytes → 16 unsigned 16-bit numbers (zero-extend). `cvtepu8_epi32`: 8 bytes → 8 int32.
`_mm256_add/sub/mullo_epi16`	16 additions / subtractions / low 16 bits of products, at once.
`_mm256_abs_epi16`, `_mm256_srli_epi16`	absolute value; logical shift right of each 16-bit element.
`_mm256_packus_epi16`	*narrow with saturation*: two registers of int16 → one of uint8, clamping each value into $[0, 255]$.
`_mm256_packs_epi32`	narrow int32 → int16 with signed saturation.
`_mm256_permute4x64_epi64`, `_mm256_permutevar8x32_epi32`	move 64-bit or 32-bit pieces *across* the two 128-bit halves (needed after the packs, see below).
`_mm256_cvtepi32_ps` / `_mm256_cvtps_epi32`	int32 ↔ float32; the float → int conversion rounds to nearest even.
`_mm256_broadcast_ss`	copy one float into all 8 lanes (used for the filter weight).
`_mm256_fmadd_ps`(a,b,c)	$a \cdot b + c$ for 8 floats, with a *single* rounding (fused multiply-add, from the FMA3 extension).

6.4 Two things that trip everyone up

6.4.1 Lanes: the 256-bit register is really two 128-bit halves

Most AVX2 integer instructions work inside each 128-bit half separately. `packus`, for example, packs the *low half of both inputs* into the low half of the output and the high halves into the high half, so the output order is scrambled relative to the input order. A *permute* afterwards puts the pixels back in order. Chapters 8 and 9 show the exact pictures; the unit tests catch any slip because the result would be garbled.

6.4.2 Tails: the image width is rarely a multiple of the vector width

A loop that handles 16 pixels per step ends with 0–15 pixels left. Options: pad the image (needs control over the input), a masked store, or handle the remainder with scalar code. We use scalar code for gray/Sobel remainders and a small temporary buffer for convolution. The tests use widths $1, 2, 3, \ldots, 15, 16, 17, \ldots, 129$ *because* every width class near 16, 32 and 64 has a different tail length.

6.5 Compiling and calling it safely

- AVX2 and FMA instructions only exist on CPUs from about 2013/2014 on. Compiling a file with `-mavx2 -mfma` lets the compiler emit them *anywhere* in that file. We therefore compile **only** `avx2.cpp` with those flags, and call into it **only after** a run-time CPU check (`__builtin_cpu_supports("avx2")`) in `dispatch.cpp`. The rest of the library runs on any x86-64 CPU.
- A global object with a constructor in `avx2.cpp` would execute at program start, before any check, and could crash an old CPU. The shuffle masks are therefore `constexpr` tables, computed by the

compiler.

- Alternative: compile everything with `-march=native`. Simpler and often slightly faster, but the binary may crash with "illegal instruction" on a different CPU model. On a cluster with several node types that is a real risk, so the Slurm scripts build per node type.

Interview soundbite

"AVX2 gives 256-bit registers: 32 bytes, 16 words or 8 floats. I compile only the intrinsics file with `-mavx2 -mfma` and gate it with a run-time CPU check. The two details I had to design around were the 128-bit lane structure (packs scramble the order, so I permute afterwards) and tails (scalar remainder, or a temporary buffer so nothing is ever written past the row)."

Chapter 7

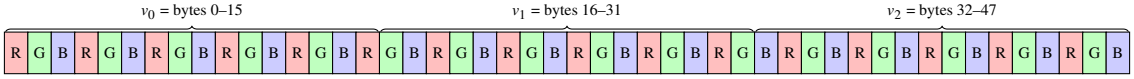
AVX2 grayscale, line by line

Reminder: $Y = (77R + 150G + 29B + 128) \gg 8$. The scalar code does this for one pixel per loop iteration. The AVX2 code converts **16 pixels per iteration**.

7.1 Problem 1: the three channels are interleaved

Memory contains `R0 G0 B0 R1 G1 B1 R2 ...`. To multiply all red values by 77 with one instruction we need 16 reds next to each other in a register, and likewise for green and blue. Getting from the interleaved layout to three separate registers is called *deinterleaving*.

Sixteen pixels are 48 bytes: exactly three 16-byte loads. Call them v_0 (bytes 0–15), v_1 (bytes 16–31) and v_2 (bytes 32–47).



The reds are at byte positions $0, 3, 6, \ldots, 45$: six of them in v_0, five in v_1, five in v_2. `pshufb` (`_mm_shuffle_epi8`) takes a register of data and a register of *indices*: output byte i is the data byte whose number is given in the index byte i; an index with its top bit set (we use `0x80`) produces a zero byte. So for each colour we build three index vectors, one per source register, which collect that colour's bytes into their final positions and leave zeros elsewhere. OR-ing the three partial results gives all 16 values of that colour in order.

	mask for v_0	mask for v_1	mask for v_2
R	0 3 6 9 12 15 – – – – – – – – – –	– – – – – 2 5 8 11 14 – – – – –	– – – – – – – – – – 1 4 7 10
G	1 4 7 10 13 – – – – – – – – – –	– – – – – 0 3 6 9 12 15 – – – – –	– – – – – – – – – – 2 5 8 11
B	2 5 8 11 14 – – – – – – – – – –	– – – – – 1 4 7 10 13 – – – – –	– – – – – – – – – – 0 3 6 9 12

output byte i = input byte at the listed index; – means index `0x80` (write 0)

The tables are built by the compiler at compile time from one formula: pixel i's colour-c byte sits at global position $g = 3i + c$, so it is read from source register $\lfloor g/16 \rfloor$ at index g mod 16.

```
struct DeinterleaveMasks {
    // m[channel][source register][destination byte] = index into that source register, or 0x80 = "
     write 0"
    std::uint8_t m[3][3][16];
};

constexpr DeinterleaveMasks makeMasks() {
    DeinterleaveMasks d{};
    for (int c = 0; c < 3; ++c)
        for (int s = 0; s < 3; ++s)
            for (int i = 0; i < 16; ++i) d.m[c][s][i] = 0x80;
    for (int c = 0; c < 3; ++c)
        for (int i = 0; i < 16; ++i) {
```

```
            const int g = 3 * i + c;              // byte position of channel c of pixel i in the
    48 bytes
            d.m[c][g / 16][i] = static_cast<std::uint8_t>(g % 16);
        }
    return d;
}
alignas(16) constexpr DeinterleaveMasks kMasks = makeMasks();
```

7.2 Problem 2: the arithmetic must not overflow

After deinterleaving, each colour is 16 bytes. We widen them to 16-bit lanes (`cvtepu8_epi16`), multiply by 77, 150, 29 (`mullo_epi16`, keeps the low 16 bits of each product) and add (with the $+128$). We showed in Chapter 3 that the largest value is $65\,408 < 65\,536$: no wrap-around, so the low 16 bits of the products are the full products and plain 16-bit adds are exact. The shift `srli_epi16(...,8)` divides by 256; it is a *logical* shift, which is right because the values are unsigned.

7.3 Problem 3: narrowing back to bytes

The 16 results are 16-bit values in $[0, 255]$ in one 256-bit register. `packus_epi16` of the low and high 128-bit halves produces one 16-byte register in the original order, which is stored with one `storeu`.

7.4 The complete kernel

```
void rgbToGrayRows(const Image& src, Image& dst, int y0, int y1) {
    const int w = src.width();
    __m128i mask[3][3];
    for (int c = 0; c < 3; ++c)
        for (int s = 0; s < 3; ++s) mask[c][s] = _mm_load_si128(reinterpret_cast<const __m128i*>(
    kMasks.m[c][s]));
    const __m256i c77 = _mm256_set1_epi16(77), c150 = _mm256_set1_epi16(150), c29 =
    _mm256_set1_epi16(29),
                  c128 = _mm256_set1_epi16(128);

    for (int y = y0; y < y1; ++y) {
        const std::uint8_t* p = src.row(y);
        std::uint8_t* q = dst.row(y);
        int x = 0;
        for (; x + 16 <= w; x += 16) {
            const std::uint8_t* s = p + 3 * x;      // reads s[0..47], inside the row because x+16
    <= w
            const __m128i v0 = _mm_loadu_si128(reinterpret_cast<const __m128i*>(s));
            const __m128i v1 = _mm_loadu_si128(reinterpret_cast<const __m128i*>(s + 16));
            const __m128i v2 = _mm_loadu_si128(reinterpret_cast<const __m128i*>(s + 32));
            const __m128i r8 = _mm_or_si128(_mm_or_si128(_mm_shuffle_epi8(v0, mask[0][0]),
    _mm_shuffle_epi8(v1, mask[0][1])),
                                            _mm_shuffle_epi8(v2, mask[0][2]));
            const __m128i g8 = _mm_or_si128(_mm_or_si128(_mm_shuffle_epi8(v0, mask[1][0]),
    _mm_shuffle_epi8(v1, mask[1][1])),
                                            _mm_shuffle_epi8(v2, mask[1][2]));
            const __m128i b8 = _mm_or_si128(_mm_or_si128(_mm_shuffle_epi8(v0, mask[2][0]),
    _mm_shuffle_epi8(v1, mask[2][1])),
                                            _mm_shuffle_epi8(v2, mask[2][2]));
            // widen 16 bytes -> 16 x uint16. 77*255 + 150*255 + 29*255 + 128 = 65408 < 65536: no
    overflow
            // in 16 bits, so the multiply-adds can be done with plain 16-bit instructions.
```

```
            __m256i acc = _mm256_mullo_epi16(_mm256_cvtepu8_epi16(r8), c77);
            acc = _mm256_add_epi16(acc, _mm256_mullo_epi16(_mm256_cvtepu8_epi16(g8), c150));
            acc = _mm256_add_epi16(acc, _mm256_mullo_epi16(_mm256_cvtepu8_epi16(b8), c29));
            acc = _mm256_add_epi16(acc, c128);
            acc = _mm256_srli_epi16(acc, 8);        // logical shift: values are 0..255 now
            const __m128i packed = _mm_packus_epi16(_mm256_castsi256_si128(acc),
    _mm256_extracti128_si256(acc, 1));
            _mm_storeu_si128(reinterpret_cast<__m128i*>(q + x), packed);
        }
        for (; x < w; ++x) {                        // 0..15 leftover pixels
            const unsigned r = p[3 * x], g = p[3 * x + 1], b = p[3 * x + 2];
            q[x] = static_cast<std::uint8_t>((77u * r + 150u * g + 29u * b + 128u) >> 8);
        }
    }
}
```

In plain words

The tail. The loop condition `x + 16 <= w` guarantees that the three loads (which read bytes $3x \ldots 3x+47$) stay inside the row: row y has $3w$ bytes and the last pixel read is $x+15 \leq w-1$. The remaining 0–15 pixels use the scalar formula. Because the test suite uses *tight* images (stride equal to the row size and an allocation that ends exactly after the last row), AddressSanitizer would flag a load one byte too far.

7.5 Why this is only a few times faster, not 16 times

Per 16 pixels the code issues 3 loads, 9 shuffles, 6 ORs, 3 widenings, 3 multiplies, 4 adds, 1 shift, 1 pack and 1 store: about 30 instructions, around 2 per pixel, compared with about 10 per pixel for scalar code. The speedup is therefore roughly $5\times$ in the instruction count, and then memory starts to limit: the kernel reads 3 bytes and writes 1 byte per pixel and has very little arithmetic to hide the memory traffic. Chapter 10 puts a number on how close it gets to the memory ceiling.

Interview soundbite

"For gray I deinterleave 16 pixels with nine `pshufb` shuffles using constexpr index tables, widen to 16-bit, do the three multiplies and the rounding add in 16-bit lanes (the maximum is 65,408, so no overflow), shift, and pack back. I verified it against all 16.7 million RGB colours. It is a memory-heavy kernel, so I compared its throughput with a memory-copy yardstick."

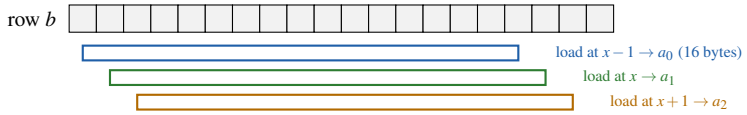
Chapter 8

AVX2 Sobel, line by line

Recall: $G_x = (a_2 - a_0) + 2(b_2 - b_0) + (c_2 - c_0)$ and $G_y = (c_0 + 2c_1 + c_2) - (a_0 + 2a_1 + a_2)$, where a, b, c are the rows above, at and below the pixel, and the index $0, 1, 2$ means column $x-1, x, x+1$.

8.1 Vectorizing across pixels

For one pixel this needs 8 neighbours. For 16 *adjacent* pixels it needs 16-byte loads from each of the three rows at three column offsets: $x-1$, x and $x+1$. Because loads may start at any address, "the same row shifted by one pixel" is just another load. That is the key trick of SIMD filtering: instead of shuffling data inside registers, load the row three times at three offsets.



Each loaded byte vector is widened to sixteen 16-bit numbers (`cvtepu8_epi16` straight from memory), because the sums exceed 255. The row in the middle (b) needs only b_0 and b_2 (its centre pixel has weight 0 in G_x and in G_y): 8 loads per 16 pixels.

8.2 The arithmetic in 16-bit lanes

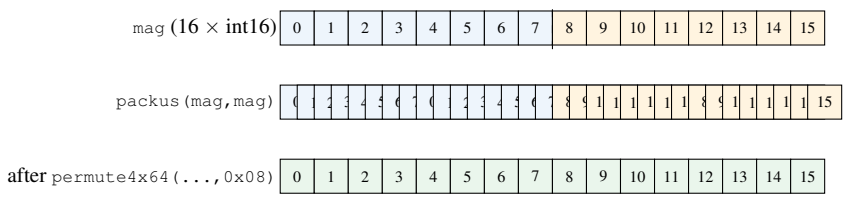
1. $G_x = (a_2 - a_0) + (c_2 - c_0) + 2(b_2 - b_0)$; doubling is an addition of the value to itself.
2. $G_y = (c_0 + c_2 + 2c_1) - (a_0 + a_2 + 2a_1)$.
3. $m = |G_x| + |G_y|$ with `abs_epi16` and `add_epi16`.

All of these fit in signed 16-bit: $|G_x|, |G_y| \le 1020$ and $m \le 2040$ (Chapter 3).

8.3 Clamping and packing for free

`packus_epi16` converts int16 to uint8 *with saturation*: negative values would become 0 and values above 255 become 255. Since $m \ge 0$, the result is exactly $\min(m, 255)$: the clamp of the formula costs no extra instruction.

But `packus` works separately inside the two 128-bit lanes:



The upper picture shows which pixel each byte holds. After `packus(mag,mag)` the low lane holds pixels 0..7 twice and the high lane pixels 8..15 twice (as 64-bit groups: [0..7], [0..7], [8..15], [8..15]). `permute4x64` with immediate `0x08` selects 64-bit group 0 and group 2 into the low 128 bits: pixels 0..15 in order. One 16-byte store writes them.

8.4 The complete kernel

```
void sobelRows(const Image& src, Image& dst, int y0, int y1) {
    const int w = src.width(), h = src.height();
    for (int y = y0; y < y1; ++y) {
        const std::uint8_t* a = src.row(y > 0 ? y - 1 : 0);
        const std::uint8_t* b = src.row(y);
        const std::uint8_t* c = src.row(y < h - 1 ? y + 1 : h - 1);
        std::uint8_t* q = dst.row(y);
        if (w == 1) {
            q[0] = sobelPixel(a, b, c, 0, 0, 0);
            continue;
        }
        q[0] = sobelPixel(a, b, c, 0, 0, 1);
        int x = 1;
        // 16 output pixels per iteration. The loads at x-1, x, x+1 read bytes up to x+16, so we
    need
        // x + 16 <= w - 1.  Everything is 16-bit: |Gx|,|Gy| <= 1020, |Gx|+|Gy| <= 2040 < 32768.
        for (; x + 16 <= w - 1; x += 16) {
            const __m256i a0 = load16(a + x - 1), a1 = load16(a + x), a2 = load16(a + x + 1);
            const __m256i b0 = load16(b + x - 1), b2 = load16(b + x + 1);
            const __m256i c0 = load16(c + x - 1), c1 = load16(c + x), c2 = load16(c + x + 1);
            const __m256i dxb = _mm256_sub_epi16(b2, b0);
            const __m256i gx = _mm256_add_epi16(_mm256_add_epi16(_mm256_sub_epi16(a2, a0),
    _mm256_sub_epi16(c2, c0)),
                                                _mm256_add_epi16(dxb, dxb));
            const __m256i top = _mm256_add_epi16(_mm256_add_epi16(a0, a2), _mm256_add_epi16(a1, a1))
    ;
            const __m256i bot = _mm256_add_epi16(_mm256_add_epi16(c0, c2), _mm256_add_epi16(c1, c1))
    ;
            const __m256i gy = _mm256_sub_epi16(bot, top);
            const __m256i mag = _mm256_add_epi16(_mm256_abs_epi16(gx), _mm256_abs_epi16(gy));
            // packus saturates int16 -> uint8, i.e. min(mag, 255) since mag >= 0.
            // It works inside each 128-bit lane, so the 16 results end up as qwords [0, 0', 1, 1'];

            // permute4x64 with 0x08 moves qwords 0 and 2 to the bottom.
            const __m256i pk = _mm256_permute4x64_epi64(_mm256_packus_epi16(mag, mag), 0x08);
            _mm_storeu_si128(reinterpret_cast<__m128i*>(q + x), _mm256_castsi256_si128(pk));
        }
        for (; x < w - 1; ++x) q[x] = sobelPixel(a, b, c, x - 1, x, x + 1);
        q[w - 1] = sobelPixel(a, b, c, w - 2, w - 1, w - 1);
    }
}
```

In plain words

Borders and tails. Rows above the first and below the last are handled by pointing the row pointers at the first/last row (replication), so the vector loop needs no special case in y. In x: pixel 0 and pixel $w-1$ need clamped column indices, so they are computed by the scalar helper; the vector loop runs over $x = 1, 17, 33, \ldots$ as long as its widest load $(x+16)$ stays within the row ($x+16 \le w-1$); the rest uses the scalar helper. A tempting off-by-one (`x + 16 <= w`) would let the last vector read one byte beyond the row *and be silently corrected* by the scalar write of the last pixel. Only the sanitizer build catches that kind of bug, which is why the test suite includes images with no padding and the `make`

`asan-test` target (Chapter 12).

Interview soundbite

"Sobel is vectorized across 16 adjacent pixels with unaligned loads of each row at offsets $-1, 0, +1$, 16-bit signed arithmetic (the maximum is 2040), absolute values, and `packus` which gives the clamp to 255 for free. `packus` works per 128-bit lane, so a `permute4x64` restores pixel order. The borders and the tail use the scalar helper, and I check that no vector load goes past the end of the row with AddressSanitizer on images with no padding."

Chapter 9

AVX2 convolution, line by line

Convolution is where AVX2 pays off the most: lots of arithmetic per pixel ($2k^2$ floating-point operations) and a very regular pattern. There are four ideas in the kernel.

9.1 Idea 1: convert each row to float once, with the border built in

The arithmetic is done in float32, but the image is bytes. Converting a byte to a float for every tap (every multiplication) would waste time: a pixel is used k^2 times. Instead, for each source row we create *one padded float copy*:

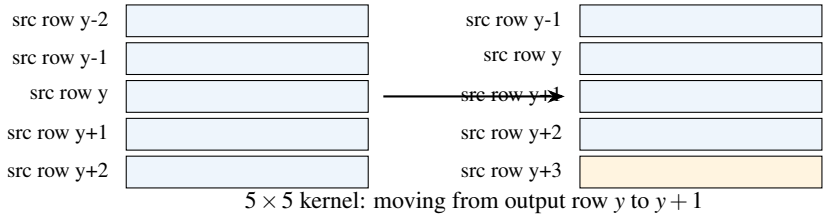
$$\text{padded}[j] = \text{src}[\text{clamp}(j-r, 0, w-1)], \qquad j = 0, \ldots, \text{round}_{64}(w) + 2r - 1.$$

The r extra entries on each side replicate the edge pixel, so the **border handling disappears from the inner loop**: every output pixel, even the first, reads k consecutive floats from the padded row. Extra entries beyond the right border let the vector loop run in full blocks of 64 pixels.

```
// Replicated-border float copy of one source row: out[j] = src[clamp(j - r, 0, w-1)], j in [0, len)
.
void convertRowPadded(const std::uint8_t* s, int w, int r, int len, float* out) {
    int j = 0;
    const float left = static_cast<float>(s[0]);
    for (; j < r && j < len; ++j) out[j] = left;
    // middle part j in [r, r+w): vectorised u8 -> float conversion, 8 pixels at a time
    int x = 0;
    for (; x + 8 <= w && r + x + 8 <= len; x += 8) {
        const __m128i b8 = _mm_loadl_epi64(reinterpret_cast<const __m128i*>(s + x));
        _mm256_storeu_ps(out + r + x, _mm256_cvtepi32_ps(_mm256_cvtepu8_epi32(b8)));
    }
    for (j = r + x; j < len; ++j) out[j] = static_cast<float>(s[clampInt(j - r, 0, w - 1)]);
}
```

9.2 Idea 2: a ring of k float rows, each converted only once

Output row y needs source rows $y-r \ldots y+r$ (k rows). Output row $y+1$ needs $y-r+1 \ldots y+r+1$: only *one new row*. The code keeps k padded rows in a small cache (slots); when moving to the next output row it converts just the one new source row into a free slot, and re-uses the rest. At the top and bottom of the image the clamped row indices repeat (row 0 is needed several times); the slot lookup by *source row number* handles this without special cases.



5×5 kernel: moving from output row y to $y+1$
reuses 4 float rows and converts 1 new one (orange).

Working set: k rows of $(w+2r)$ floats. For $w = 4096$ and $k = 9$ that is $9 \cdot 4104 \cdot 4$ bytes ≈ 148 KB, which fits in the L2 cache of the CPUs we use (Chapter 10). This is the **cache-aware** part of the design: the image is streamed once from memory, and everything that is re-used lives in the cache.

9.3 Idea 3: eight accumulators in registers, one broadcast weight

For a block of 64 adjacent output pixels (8 vectors of 8 floats) the kernel keeps **8 accumulators in registers**, loops over the k^2 taps, and for each tap does

$$\text{acc}_v \mathrel{+}= w_{ji} \cdot \text{padded}_j[x+i+8v], \qquad v = 0,\ldots,7,$$

using `broadcast_ss` to copy the weight into all lanes and `fmadd_ps` to multiply-add. The loads at offset $+i$ are, again, plain unaligned loads of the same row shifted by one float.

Why 8 accumulators? An FMA has a *latency* of about 4 cycles (the result is not ready for 4 cycles) but a modern core can *start* two per cycle. If each FMA depends on the previous one (a single accumulator), the core waits 4 cycles for each; with 8 independent accumulators it can keep both FMA units busy. The ablation with 4 accumulators measures the difference (Chapter 13).

In plain words

Registers vs. memory. Because the accumulators stay in registers for all k^2 taps, the only memory traffic in the inner loop is reading the padded rows (served by the cache). The alternative that a compiler can produce from the "row accumulate" loop nest, `acc[x] += w * row[x+i]` over a whole row, reads and writes the accumulator array in memory for *every* tap. `ablation::convolveAutoVecRowAcc` is exactly that, and the measured gap to the register version is large.

9.4 Idea 4: the same order of operations as the scalar code

Every output pixel is accumulated starting from 0, with taps in the order $j = 0,\ldots,k-1$ (outer) and $i = 0,\ldots,k-1$ (inner). The vector code does the same for its 64 pixels in parallel; vectorizing across pixels changes which pixels are computed together, but never the order of operations *within* one pixel. As a result:

- AVX2 is bit-identical to a scalar `fmaf` loop in the same order (the reference), for every kernel and image: tested.
- The scalar and autovec versions are bit-identical to each other (multiply, then add).
- Differences between the two families are possible only because one fuses the multiply-add (one rounding) and the other does not (two roundings). Chapter 11 analyses that.

9.5 Converting back to bytes

After the taps, each accumulator holds 8 float sums. `cvtps_epi32` rounds them to nearest, ties to even (the CPU's default mode, the same rule as the scalar `lrintf`); `packs_epi32` narrows int32 to int16 and `packus_epi16` narrows to uint8, *both with saturation*, which implements the clamp to $[0,255]$. As in the Sobel kernel the two-lane structure scrambles the order, here in units of 4 pixels; one `permutevar8x32` with the index vector $(0,4,1,5,2,6,3,7)$ restores it:

after `packus`, 32-bit group:	0	1	2	3	4	5	6	7
contains pixels:	0–3	8–11	16–19	24–27	4–7	12–15	20–23	28–31
index vector picks group:	0	4	1	5	2	6	3	7
so the output contains pixels:	0–3	4–7	8–11	12–15	16–19	20–23	24–27	28–31

```
// Turns 4 vectors of 8 int32 (pixels 0-7, 8-15, 16-23, 24-31) into 32 saturated bytes in pixel
    order.
inline __m256i packInt32To32Bytes(__m256i c0, __m256i c1, __m256i c2, __m256i c3) {
    const __m256i p01 = _mm256_packs_epi32(c0, c1);    // int32 -> int16 (saturating), per 128-bit
    lane
    const __m256i p23 = _mm256_packs_epi32(c2, c3);
    const __m256i p = _mm256_packus_epi16(p01, p23);   // int16 -> uint8 (saturating) => clamp to
    0..255
    // dwords now are [px0-3, px8-11, px16-19, px24-27 | px4-7, px12-15, px20-23, px28-31]
    return _mm256_permutevar8x32_epi32(p, _mm256_setr_epi32(0, 4, 1, 5, 2, 6, 3, 7));
}
```

9.6 The complete kernel

```
// NV = number of 8-float vectors per block (block = 8*NV output pixels). 8 independent
    accumulators hide the
// 4-cycle FMA latency on CPUs that can start two FMAs per cycle; 4 is the ablation.
template <int NV>
void convolveRowsImpl(const Image& src, Image& dst, const Kernel& K, int y0, int y1) {
    constexpr int kBlock = 8 * NV;
    const int w = src.width(), h = src.height(), k = K.size, r = k / 2;
    const float* wt = K.w.data();
    const int wRound = (w + kBlock - 1) / kBlock * kBlock;
    const int RL = wRound + 2 * r;                // length of one padded float row

    // k slots of padded float rows; slotRow[s] = which source row currently lives in slot s
    std::unique_ptr<float, FreeDeleter> ring(static_cast<float*>(alignedAlloc(sizeof(float) *
    static_cast<std::size_t>(RL) * k, 64)));
    int slotRow[kMaxKernelSize];
    for (int s = 0; s < k; ++s) slotRow[s] = -1;

    for (int y = y0; y < y1; ++y) {
        int ids[kMaxKernelSize];
        const float* rowp[kMaxKernelSize];
        for (int j = 0; j < k; ++j) ids[j] = clampInt(y - r + j, 0, h - 1);
        for (int j = 0; j < k; ++j) {
            int slot = -1;
            for (int s = 0; s < k; ++s)
                if (slotRow[s] == ids[j]) { slot = s; break; }
            if (slot < 0) {                        // not cached: reuse a slot that no needed row
    lives in
                for (int s = 0; s < k && slot < 0; ++s) {
                    bool needed = false;
                    for (int t = 0; t < k; ++t) needed = needed || (slotRow[s] == ids[t]);
                    if (!needed) slot = s;
                }
                convertRowPadded(src.row(ids[j]), w, r, RL, ring.get() + static_cast<std::size_t>(
    slot) * RL);
                slotRow[slot] = ids[j];
            }
            rowp[j] = ring.get() + static_cast<std::size_t>(slot) * RL;
        }

        std::uint8_t* q = dst.row(y);
```

```
        for (int x = 0; x < w; x += kBlock) {
            __m256 acc[NV];
            for (int v = 0; v < NV; ++v) acc[v] = _mm256_setzero_ps();
            for (int j = 0; j < k; ++j) {
                const float* p = rowp[j] + x;
                const float* wk = wt + j * k;
                for (int i = 0; i < k; ++i) {
                    const __m256 wv = _mm256_broadcast_ss(wk + i);
                    for (int v = 0; v < NV; ++v) acc[v] = _mm256_fmadd_ps(wv, _mm256_loadu_ps(p + i
     + 8 * v), acc[v]);
                }
            }
            // round to nearest even (MXCSR default), pack with saturation, store
            __m256i out[NV / 4];
            for (int g = 0; g < NV / 4; ++g)
                out[g] = packInt32To32Bytes(_mm256_cvtps_epi32(acc[4 * g]), _mm256_cvtps_epi32(acc
    [4 * g + 1]),
                                            _mm256_cvtps_epi32(acc[4 * g + 2]), _mm256_cvtps_epi32(
    acc[4 * g + 3]));
            if (x + kBlock <= w) {
                for (int g = 0; g < NV / 4; ++g) _mm256_storeu_si256(reinterpret_cast<__m256i*>(q +
     x + 32 * g), out[g]);
            } else {                                // last, partial block: never write past the row
                alignas(32) std::uint8_t tmp[kBlock];
                for (int g = 0; g < NV / 4; ++g) _mm256_store_si256(reinterpret_cast<__m256i*>(tmp +
     32 * g), out[g]);
                std::memcpy(q + x, tmp, static_cast<std::size_t>(w - x));
            }
        }
    }
}
```

In plain words

The last block of a row. If fewer than 64 pixels remain, the kernel still computes 64 pixels (the extra ones read replicated padding, which is defined data) but stores into a small temporary array and copies only the valid bytes (`memcpy(q + x, tmp, w - x)`). It never writes past the end of the row, which the padding test and the sanitizer check.

Guard on the kernel weights. `cvtps_epi32` returns the value `0x80000000` for numbers outside the int32 range, which would turn a huge positive sum into 0 after the pack instead of 255. `Kernel::validate()` therefore rejects kernels with $255\sum|w| > 10^9$ and non-finite weights, so scalar and AVX2 cannot diverge on overflow.

Interview soundbite

"The AVX2 convolution converts each source row to a padded float row once, keeps k of them in a small ring so the next output row reuses $k-1$ of them, and computes 64 output pixels per block with eight register accumulators and `fmadd`. Borders disappear because the padding is built into the row copy. Per pixel it accumulates in the same order as the scalar code, so it is bit-identical to an FMA reference; I tested that for 7 kernel sizes, widths around every block boundary, and several thread counts."

Chapter 10

Memory, caches and the roofline

Chapters 7–9 made the arithmetic cheap. Once arithmetic is cheap, the question becomes: *can the memory system deliver the pixels fast enough?* This chapter is the vocabulary and the measurements for that question.

10.1 The memory hierarchy in one picture

A CPU core cannot compute on data that is far away. Between the core and main memory (DRAM) sit small, fast memories called **caches**:

level	typical size (per core / shared)	what it means for us
registers	16 vector registers of 32 bytes	the accumulators of the convolution live here
L1 data cache	tens of KB per core	a few image rows; latency of a few cycles
L2 cache	about 1–2 MB per core on current server CPUs	the k float rows of the convolution ring
L3 cache	tens of MB, shared by the cores of a socket	small and medium images
main memory	many GB, shared	large images; the bandwidth is the limit

The numbers differ per CPU. Never quote them from memory in an interview; print them with `lscpu` on the machine you measured on (the Slurm scripts store this in `meta.txt`). The sandbox used for the example numbers reports `L1d 96 KiB (2 instances), L2 4 MiB (2 instances)`, i.e. 48 KB and 2 MB per core, and a very large shared L3 that is a property of the virtual machine host. For the cluster partitions the bwHPC wiki lists the CPU types; the cache sizes of those models should be read from `lscpu` in your own job output rather than assumed.

Data moves between levels in **cache lines** of 64 bytes. Reading one byte brings in 64. This is why the library aligns rows to 64 bytes and why the vector loops are written to walk through memory in order: the *hardware prefetcher* recognises a sequential stream and loads the next lines before the program asks for them.

In plain words

Library analogy. The shelf next to your desk (L1) holds a few books. The room's shelves (L2/L3) hold many. The city library (DRAM) holds everything but every trip takes long. Good code reads a book once, uses everything in it, and puts it back. Bad code goes back to the library for every sentence.

10.2 Working sets of the three operations

The **working set** is the data that is re-used before it is no longer needed.

- **Gray**: nothing is re-used. Each input byte is read once, each output byte written once. The working set is a few lines. It is a pure *streaming* kernel: speed = how fast bytes flow.

- **Sobel**: each source row is used by three consecutive output rows. The three rows are read in the same pass; the working set is three rows (a few KB to a few tens of KB, e.g. $3 \cdot 8192$ bytes). It fits in L1/L2 for any realistic width, so every source byte comes from DRAM *once*. Streaming again.
- **Convolution**: k float rows of $w+2r$ entries, i.e. $4k(w+2r)$ bytes: 148 KB for $w = 4096$, $k = 9$ (Chapter 9). That is bigger than L1 but fits in L2. The design keeps this ring in cache and streams the image once.

10.3 Arithmetic intensity and the roofline

Define the **arithmetic intensity** $I = \text{operations}/\text{bytes moved}$. A processor has two ceilings:

$$\text{attainable performance} \;=\; \min\bigl(P_{\text{peak}},\; B \cdot I\bigr),$$

where P_{peak} is the peak arithmetic rate and B the memory bandwidth. Plot performance against I: a sloped line on the left (memory-bound: more intensity is directly more speed) and a flat roof on the right (compute-bound). This is the **roofline model**.

- **Gray and Sobel** do a handful of integer operations per pixel and move 2–4 bytes per pixel: far left, memory-bound. Once SIMD has removed the instruction overhead, the next ceiling is the memory system.
- **Convolution** does $2k^2$ floating-point operations per pixel for the same 2 bytes of traffic: $I = k^2$ operations per byte. Already for $k = 3$ it is far to the right of the other two, so it is compute-bound and its limit is the FMA units.

10.3.1 A measured memory yardstick

To avoid guessing B, the benchmark has an operation `copy`: a multi-threaded `memcpy` of the same pixel rows into another image. A kernel cannot move bytes faster than a plain copy does, so its *minimum traffic rate* (input bytes read plus output bytes written, counted once) should be compared with the copy's.

image	threads	copy (GB/s)	gray AVX2	Sobel AVX2
2048^2	1	24.6	14.5 (59 %)	8.0 (33 %)
2048^2	2	49.2	27.5 (56 %)	15.6 (32 %)
4096^2	1	25.0	14.6 (58 %)	8.4 (34 %)
4096^2	2	50.4	27.7 (55 %)	16.3 (32 %)

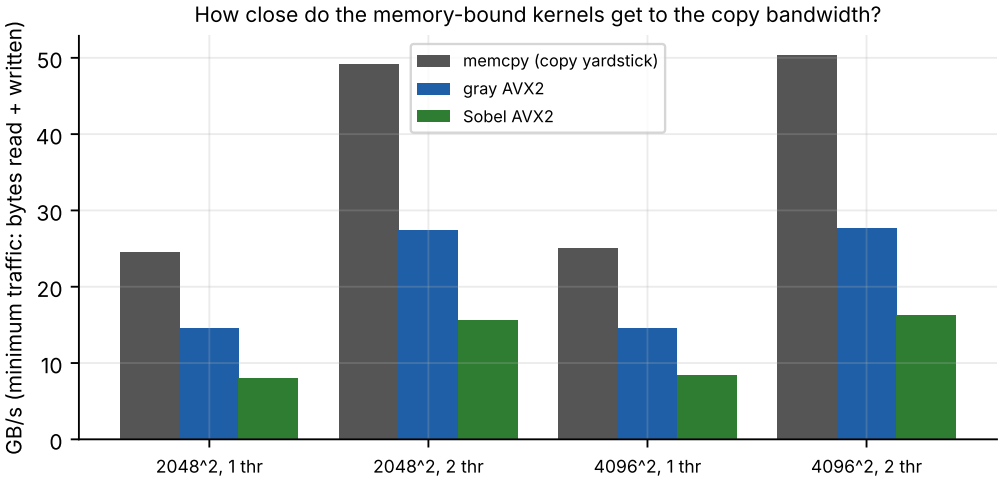


Figure 10.1: Minimum-traffic rate of the AVX2 kernels compared with a plain copy of the same pixels (sandbox VM).

How to read this honestly:

- On this machine one thread copies about 25 GB/s from DRAM (images of 2048^2 and larger) and two threads about 50 GB/s, so the sandbox did not saturate its memory with two threads. For small images that fit in cache the copy is faster (65 GB/s at 512^2), which is the cache hierarchy showing.
- AVX2 gray reaches about 58 % of the copy rate at 4096^2, Sobel about 34 %. The copy rate counts a read and a write of one byte per pixel; gray reads three bytes per pixel and Sobel reads each pixel through three rows, so neither is expected to reach 100 % of that figure with a one-pass design. The conclusion is not "at the roof", it is: *within a factor 2–3 of the memory ceiling*, which is where a SIMD kernel that does a little work per byte should be, and far from the instruction-bound regime of the scalar code.
- This is a measurement on a VM. The same experiment on the cluster (E5 in the scripts) gives *your* yardstick; a different number there is expected and fine.

10.3.2 Compute side: how close is convolution to the FMA roof?

The theoretical peak of one core with two 256-bit FMA units is 2 FMA $\times$ 8 lanes $\times$ 2 flop $=$ 32 flop/cycle. At the nominal 2.1 GHz of the sandbox CPU that is 67 GFLOP/s, and the AVX2 convolution reaches up to 58 GFLOP/s, about 86 % of that figure. Two caveats keep this honest: (1) the clock under load may differ from the nominal value (turbo, or a lower AVX frequency), so the percentage is a rough indicator, not a precise efficiency; (2) the count $2k^2$ flop per pixel counts every tap as one multiply and one add, whether or not the zero weights could be skipped. What the number shows qualitatively: the kernel is limited by the FMA units, not by loads or by the memory system, which is exactly what the design of Chapter 9 aims for. Whether the cluster CPUs (Ice Lake, Zen 4) show the same fraction is a measurement for you to make; do not assume it.

10.4 Image size and the cache

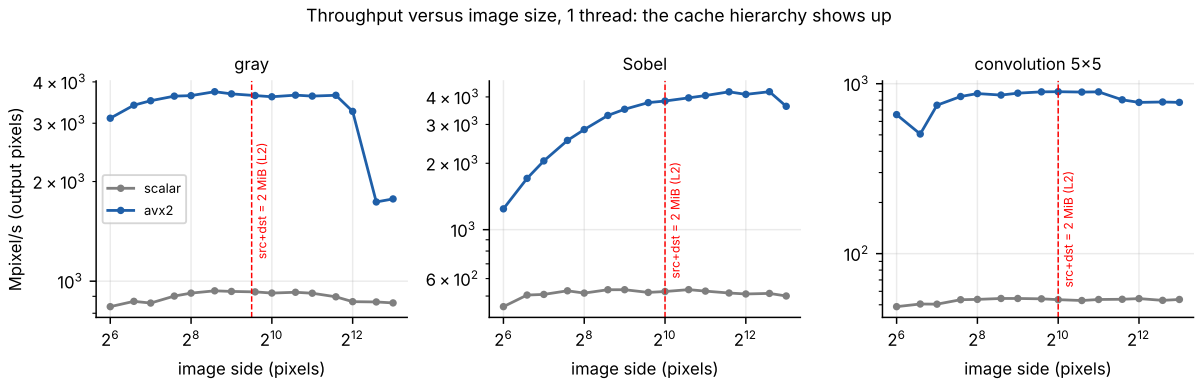


Figure 10.2: Throughput of scalar and AVX2 versus image side (1 thread). The dashed red line marks the size at which source plus destination together (2 bytes per pixel, 4 for gray) equal 2 MiB, the per-core L2 of this VM.

Observations in the sandbox data, with the explanation they support:

- **Scalar** throughput is flat: it is limited by instructions, and the memory system is never the bottleneck, so image size does not matter.

- **AVX2 gray** falls from about 3680 Mpix/s at 512^2 to about 1772 Mpix/s at 8192^2 once the data come from DRAM: the SIMD kernel is fast enough to feel the memory.
- **AVX2 Sobel** is much less sensitive (3516to 3626 Mpix/s): it does more arithmetic per byte, so it stays closer to compute-bound.
- **Very small images** (below about 100 pixels per side) lose speed to fixed per-call costs: the vector loop only starts after the first pixel and ends before the last, so a larger share of pixels takes the scalar border path.

10.5 Does the row stride matter?

Chapter 2 explained why the library avoids strides that are an exact multiple of 4096 bytes: addresses 4096 bytes apart map to the same cache set, so k rows that are read together can evict each other (*cache associativity conflict* and, on some CPUs, *4K aliasing* between loads and stores). Sobel reads three rows and writes one at the same column offset: that is the pattern that would suffer.

The experiment compares the same operation with stride = width (tight) and stride = width +64. Strides of 4096 and 8192 bytes (widths 4096 and 8192) are exact page multiples; widths 1024 and 2048 are powers of two but not page multiples and serve as controls. The last six rows use a 4096-pixel-wide, 512-row image with large kernels.

width	case	stride = width (ms)	stride +64 (ms)	power-of-two slower by
1024	sobel scalar	2.17	2.08	+4.6 %
1024	sobel avx2	0.28	0.28	+0.1 %
1024	conv avx2 k=9	3.30	3.28	+0.6 %
2048	sobel scalar	8.60	8.99	-4.3 %
2048	sobel avx2	1.07	1.20	-11.0 %
2048	conv avx2 k=9	14.05	13.99	+0.4 %
4096	sobel scalar	38.18	32.82	+16.3 %
4096	sobel avx2	4.40	3.99	+10.4 %
4096	conv avx2 k=9	56.25	55.43	+1.5 %
8192	sobel scalar	137.21	130.16	+5.4 %
8192	sobel avx2	18.54	18.42	+0.7 %
8192	conv avx2 k=9	222.52	222.11	+0.2 %
4096×512	conv scalar k=15	327.47	314.82	+4.0 %
4096×512	conv autovec k=15	365.18	366.21	-0.3 %
4096×512	conv avx2 k=15	17.44	17.49	-0.3 %
4096×512	conv scalar k=21	680.57	637.85	+6.7 %
4096×512	conv autovec k=21	693.49	699.34	-0.8 %
4096×512	conv avx2 k=21	33.03	32.74	+0.9 %

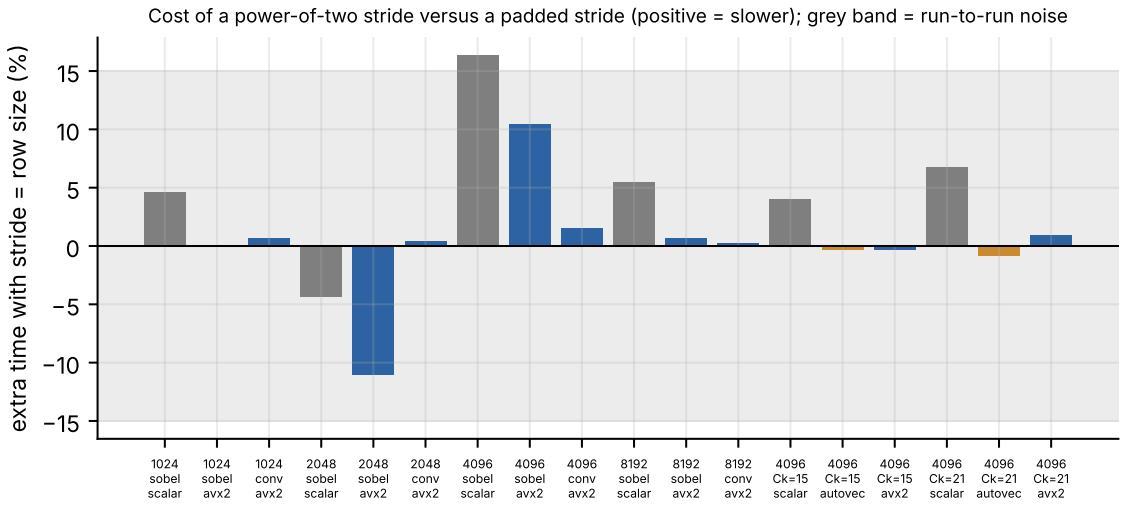


Figure 10.3: Relative cost of the page-multiple stride. Positive values mean the padded stride is faster; the grey band is the run-to-run spread measured in E6 (at least ±5 %).

What this experiment does and does not show

The effect is small and inconsistent: it ranges from about −11 % to +16 % (positive: the page-multiple stride is slower), and the run-to-run spread on this VM is of the same order (Chapter 13). The one result in the direction the theory predicts is Sobel at width 4096 (scalar +16 %, AVX2 +10 %); but width 8192, which is also a page multiple, shows only +5 % and +0.7 %, width 2048 (not a page multiple) shows the *opposite* sign, and convolution, whose working set is the float ring rather than the image rows, shows no effect beyond ±1.5 % for AVX2. So the honest statement is: *"the padding costs 1.6 % memory and removes a failure mode that the theory predicts; on this VM I could not show an effect distinguishable from run-to-run noise."* Do not claim a measured gain. Repeat E3 on the cluster (the script does), and report what you see, including a null result.

10.6 Other effects the code does not exploit (and you can mention)

- **Tiling / loop blocking over several output rows** would let each loaded source row be used by more than one output row in registers (the convolution here reloads each float row once per output row that uses it: k times). A 2× or 4× row blocking reduces loads; it is the next step toward the FMA roof.
- **Non-temporal stores** (`_mm256_stream_si256`) skip polluting the cache with output that will not be read again; they help bandwidth-bound writes of large images and hurt small ones.
- **NUMA and first touch.** On a two-socket node, memory is attached to a socket. An image allocated and first written by one thread lives near that thread's socket; other sockets read it over a slower link. The scaling script pins threads close together to stay in one place. A full treatment would initialise the image in parallel with the same band layout ("first touch").
- **Huge pages** (2 MB) reduce the number of address translations (TLB misses) for large images.

Interview soundbite

"The three kernels sit at very different places on the roofline. Gray and Sobel stream: after vectorization their limit is memory bandwidth, and I measured how close they get with a memcpy of the same pixels. Convolution has k^2 operations per byte, so it is bound by the FMA units; the ring of float rows keeps the re-used data in L2, and I measured it at roughly 86 percent of the nominal peak of one core on my

test machine *[yours: on the cluster core]*."

Chapter 11

Numerics: what "the same result" means

A SIMD library is only useful if its output can be trusted. This chapter defines exactly what is promised for each operation, and why the promise for convolution has to be stated more carefully than for the other two.

11.1 Integer operations: exact

Gray and Sobel use integers only. Integer addition and multiplication are exact as long as the numbers fit in the type; the proof that they fit is a short calculation (Chapter 3):

- gray: $77R + 150G + 29B + 128 \le 255 \cdot 256 + 128 = 65\,408 < 2^{16}$, so unsigned 16-bit lanes never wrap. The test `gray_exhaustive_all_16777216_colours` does not rely on the proof alone: it feeds *every one* of the 2^{24} possible colours through the AVX2, the scalar and the reference code and compares.
- Sobel: $|G_x|, |G_y| \le 1020$, $|G_x| + |G_y| \le 2040 < 2^{15}$, so signed 16-bit lanes are enough.

Because there are no rounding errors, every implementation must give **identical bytes**, and every test of these two operations demands zero difference.

11.2 Floating point in one page

A 32-bit float stores a number as $\pm m \cdot 2^e$ with a 24-bit significand m. Consequences:

- Not every number is representable: 0.1 is not. Each arithmetic result is **rounded** to the nearest representable value. The relative error of one rounding is at most $u = 2^{-24} \approx 6 \cdot 10^{-8}$ ("unit roundoff").
- Addition is **not associative**: $(a+b)+c$ and $a+(b+c)$ can differ in the last bit. The order of the operations is part of the definition of the result.
- A **fused multiply-add** (FMA) computes $a \cdot b + c$ with *one* rounding instead of two. It is faster and slightly more accurate, but gives a *different* last bit than multiply-then-add.

In plain words

Think of rounding as measuring with a ruler that has a mark every millimetre. Add up a thousand lengths, rounding after every addition, and the total can be a millimetre or two off, and the *size* of that error depends on the order in which you added. Nobody is wrong; the order is simply part of the recipe.

11.3 The convolution contract

For one output pixel the library computes

$$s = \sum_{j=0}^{k-1} \sum_{i=0}^{k-1} w_{ji}\, p_{ji}, \qquad \text{out} = \text{clamp}\big(\text{rne}(s), 0, 255\big),$$

with the summation order **fixed**: j outer, i inner, starting from 0. "rne" is round to nearest, ties to even (the default IEEE mode: $0.5 \to 0$, $1.5 \to 2$, $2.5 \to 2$). Two families of implementations follow this recipe, differing in one thing only:

family	each step is	produced by
FMA family	$\text{acc} \leftarrow \text{fma}(w, p, \text{acc})$ (one rounding)	AVX2 kernels and `reference::convolveFmaFloat`
two-rounding family	$\text{acc} \leftarrow \text{fl}(\text{acc} + \text{fl}(w \cdot p))$	scalar, autovec, row-accumulate rewrite

The build flags make this explicit: the scalar and autovec translation units are compiled with `-ffp-contract=off`, which forbids the compiler from silently turning multiply-add pairs into FMAs. Without that flag a build with `-march=native` would change the numerical results of "scalar" behind the programmer's back, and the claim "these two are bit-identical" would hold on one machine and fail on another.

What the tests then establish, in the order of strength:

1. **AVX2 = FMA reference, bit for bit** (`conv_avx2_is_bit_identical_to_the_fma_reference`): the compared quantity is the output byte of every pixel for random kernels with positive and negative weights, many sizes, strides and thread counts. The reference is written independently with `fmaf` in the same order.
2. **scalar = autovec = row-accumulate rewrite, bit for bit**: same operations, same order, no FMA.
3. **Float32 versus exact arithmetic**: an implementation can differ from the exact result (computed in double precision, ties to even) only when the exact value lies so close to a rounding boundary ($x.5$) that the float rounding error moves it across. The test `conv_scalar_differs_from_exact_rounding_only_next_to_ties` checks that every such difference occurs within the proven error bound of a tie, and never exceeds one grey level. The independent NumPy/SciPy cross-check (float64, Chapter 12) measured 0.00070 % of pixels differing by one grey level for box and Gaussian kernels over all three implementations, never by more. The FMA family and the two-rounding family can therefore differ from each other by one level on such near-ties; they cannot differ otherwise.
4. **Exact kernels, zero tolerance** (`conv_binomial_kernels_are_exact_in_every_implementation`): for binomial kernels up to 9×9 the 2D weights are integer multiples of 2^{-16} at most, every product has fewer than 24 significant bits and every partial sum is a multiple of 2^{-16} below 2^8, i.e. it needs at most $8 + 16 = 24$ bits. Nothing is ever rounded, the order does not matter, and *all* implementations including the FMA family must agree exactly. This is the test that proves the data movement (shuffles, lane order, border logic) is right independently of any rounding argument.

11.4 An error bound that is tested, not just stated

For a sum of $n = k^2$ terms in float arithmetic with FMA, the standard result is

$$\left|\hat{s} - s\right| \;\le\; \gamma_n \sum |w_{ji} p_{ji}| \;\le\; \gamma_n \cdot 255 \sum |w_{ji}|, \qquad \gamma_n = \frac{nu}{1 - nu}.$$

The test `conv_float_error_is_within_the_theoretical_bound` computes the same sum in double precision (`reference::convolveDouble`, same order) and compares: the largest observed error was 0.092 times the bound. In other words the bound holds with an order of magnitude to spare, as expected: it describes the worst case in which all rounding errors have the same sign and the largest size, which real data never does. In practice this means a float32 result has an error far below $\frac{1}{2}$ grey level, and a mismatch in the output byte can occur only where the exact value is within about 10^{-4} of a rounding boundary.

11.5 Rounding and clamping details that cause real bugs

- **Round-half-even versus half-up.** The scalar code uses `lrintf`, the AVX2 code `cvtps_epi32`; both use the processor’s rounding mode (default: nearest-even). A kernel that rounds “half up” differs on every exact tie, so ties are not a theoretical curiosity: any kernel with power-of-two weights, such as the binomial filters, produces exact $x.5$ sums on integer pixels all the time. The mutation *“truncate instead of round”* is killed by the tests.
- **Clamp after rounding, not before.** A value of 255.4 must give 255, not 256 clamped later by wrap-around. The AVX2 path uses saturating packs (`packs_epi32`, `packus_epi16`), which clamp to $[-32768, 32767]$ and then $[0, 255]$. This is only correct if the 32-bit integer is itself correct, which needs the next point.
- **Overflow in the float-to-int conversion.** `cvtps_epi32` returns the “integer indefinite” value -2^{31} for inputs outside the 32-bit range, which would turn a huge positive sum into the minimum and clamp it to 0, silently. `Kernel::validate` therefore rejects kernels where $255 \sum |w| > 10^9$ and any kernel containing NaN or infinity; the test `bad_arguments_are_rejected` covers it.
- **Negative weights.** Sharpening and emboss kernels produce negative sums; those clamp to 0. The test kernels include random weights in $[-1, 1]$ so both clamp directions are exercised.
- **The sum of the weights.** A blur must have $\sum w = 1$ to preserve brightness; the Gaussian factory normalises in double and stores float. The test `conv_blur_of_constant_image_is_constant_and_brightness_is_kept` checks that a constant image stays constant.

11.6 OpenCV and the 14-bit gray conversion

OpenCV’s `cvtColor` computes gray with 14-bit coefficients ($0.299, 0.587, 0.114$ scaled by 2^{14}), the library with 8-bit coefficients ($77, 150, 29$ over 256). The results differ by up to 1 grey level on a minority of pixels (7.43 % on the test image). This is a *design difference*, not a bug of either library, and Chapter 13 reports the comparison with that caveat. If an interviewer asks, the right statement is: “8-bit coefficients keep the intermediate in 16 bits, so one 256-bit register processes 16 pixels; the price is a coefficient error of about 0.2%, and the output differs from the 14-bit conversion by at most one grey level.”

Interview soundbite

“For gray and Sobel the result is integer, so all implementations are bit-identical, and I checked the gray conversion for all 16.7 million colours. For convolution I fixed the accumulation order and compiled the scalar path with FMA contraction off. The AVX2 result equals an independent `fmaf` reference bit for bit; against a float64 SciPy reference a one-level difference occurs only on near-ties, in about seven of a million pixels for box and Gaussian kernels, and binomial kernels, which are exact in float, agree with zero tolerance. The float error stays below a tenth of the theoretical bound.”

Chapter 12

Testing: how we know it is correct

Optimised code is dangerous in a specific way: it is full of tricks (shuffles, lane permutes, partial vectors, ring buffers), and a wrong trick often produces output that *looks* right. A test suite for such code has to be designed against the failure modes of SIMD code, not just against "does it run". This chapter lists the layers, what each is for, and what the numbers mean.

12.1 The layers

layer	what it checks (and the test that does it)
hand-computed examples	tiny images whose result a human can verify on paper: Sobel of a step edge, gray of primary colours. `gray_known_values`, `sobel_hand_computed_examples`
exhaustive	all 2^{24} colours through gray, no sampling. `gray_exhaustive_all_16777216_colours`
properties	statements that follow from the maths and need no reference: Sobel of a constant image is 0; transposing the input transposes the output; inverting the input leaves the output unchanged; a delta kernel shifts the image; the identity kernel returns the input; blur keeps constants. `sobel_symmetries_hold`, `conv_delta_kernel_shifts_the_image`, ...
reference equivalence	optimised versus a plain, obviously correct implementation over many sizes (22 widths from 1 to 129 chosen around the vector sizes 16, 32 and 64), strides, image contents and thread counts (1 to 8 for gray and Sobel; up to 200 in the dedicated thread test). *the* `..._matches_reference_...` *tests*
bit exactness	AVX2 = FMA reference; scalar = autovec; exact (binomial) kernels with zero tolerance. `conv_avx2_is_bit_identical_to_the_fma_reference`, `conv_binomial_kernels_are_exact_in_every_implementation`
numerical bound	observed float error versus the proven bound. `conv_float_error_is_within_the_theoretical_bound`
memory safety	images with no padding at all, run under AddressSanitizer; the padding bytes must be untouched. `tight_images_without_any_padding`, `outputs_never_touch_the_row_padding`, `make asan-test`
API contract	mismatched sizes, overlapping source and destination, even kernels and NaN weights must throw. `bad_arguments_are_rejected`
independent implementation	a second implementation by other libraries, in another language: NumPy and SciPy (and OpenCV where installed). `python/crosscheck.py`
tests of the tests	seed one deliberate bug at a time and demand that the suite fails. `scripts/mutation_test.sh`

The suite has 24 test groups and executes 4 269 710 individual checks (a check is one comparison of a pixel, a size or a property); the last run reported 0 failures. The check count is dominated by the exhaustive gray test and the pixel-by-pixel comparisons; it is not a quality metric on its own, and the guide does not present it as one.

12.2 Why the edge sizes matter most

A vector loop processes V pixels at a time (16 for gray and Sobel, 64 for the convolution block). The tail of the row, $w \bmod V$ pixels, is handled by a different code path. Almost all real SIMD bugs live in these paths: the last vector reads past the end, the tail is skipped, the block is partially written. So the tests are

parameterised by width: a list of widths from 1 to 129 clustered around 16, 32, 64 (for example 15, 16, 17, 18, 19, 31, . . . , 65, 66), heights 1, 2, 3 and larger (the border logic in y is different for images with fewer rows than the kernel), padded and unpadded strides.

12.3 The memory-safety trap: a bug the output cannot show

Consider this bug in the Sobel kernel: the vector loop condition is `x + 16 <= w` instead of `x + 16 <= w - 1`. The last vector then loads one byte beyond the row. The computed value of the last pixel is wrong, but the scalar code that finishes the row *overwrites it with the right value*. Output identical, test passes, code wrong: on an image whose last row is followed by an unmapped page the program crashes, and on a threaded run it reads a byte that another thread is writing.

How the suite catches it:

1. The image allocator requests *exactly* the number of bytes needed (via `posix_memalign`, not rounded up), so there is no hidden slack after the last row;
2. `tight_images_without_any_padding` uses images whose stride equals their width;
3. AddressSanitizer instruments every load; a one-byte over-read is reported as an error with the exact line.

Step 1 was discovered by the mutation test: the first version of the allocator rounded sizes up to a multiple of 64 and hid the over-read, so the seeded bug *survived*. This is exactly the purpose of mutation testing, and the fix (exact allocation) is now part of the library.

12.4 Mutation testing: a measurement of the test suite

A test that has never failed has never been shown to work. `scripts/mutation_test.sh` applies 12 small, realistic bugs, one at a time, rebuilds, runs the tests and records whether the suite noticed. A bug that goes unnoticed (a *survivor*) points at a missing test.

seeded bug	result
Sobel: drop the weight 2 of the middle row	killed (tests)
gray: green coefficient 150 -> 151	killed (tests)
Sobel: vector loop reads one byte past the row (result silently repaired by the scalar tail)	killed (AddressSanitizer)
conv: copy one byte too many for the last partial block (writes into row padding)	killed (tests)
conv: ring cache may evict a row that is still needed	killed (tests)
conv: omit the lane-order fix after packus	killed (tests)
conv: truncate instead of round (first vector of each group)	killed (tests)
scalar Sobel: wrong right-border neighbour	killed (tests)
scalar conv: clamp at 254 instead of 255	killed (tests)
scalar gray: rounding constant 128 -> 127	killed (tests)
OpenMP: last band loses its final row	killed (tests)
Image: remove the page-multiple stride guard	killed (tests)

12 of 12 mutants were killed in the final run. Be exact about what that claim covers: twelve hand-picked bugs of the kinds that occur in this code base (wrong constant, off by one, wrong lane order, cache eviction, lost last row, . . .). It is not a statistical coverage figure, and a tool-generated mutation score over thousands of mutants would be a different, stronger statement.

12.5 The independent cross-check

Comparing the optimised code only with the library's own scalar code has a blind spot: if the scalar code and the reference share a misunderstanding of the specification (for example, which way Sobel's signs go, or whether the border is replicated or mirrored), every test passes. `python/ref_numpy.py` implements the same three operations from the specification using NumPy and `scipy.ndimage.correlate` (float64, `mode='nearest'` for the replicated border, `np.rint` for ties to even). The script `crosscheck.py` runs the command-line tool for each implementation and thread count on the synthetic and the real test images, and compares:

- gray and Sobel, and convolution with exact kernels (identity, binomial, sharpen, emboss): must be **identical**;
- box and Gaussian kernels (whose float weights are not exact): at most one grey level of difference.

The last run made 786 comparisons with 0 mismatches; of the pixels from the non-exact kernels, 0.00070 % differed by one level and none by more (the largest difference observed is 1).

Limits of this suite

Not covered, deliberately or for lack of time: fuzzing with a coverage-guided tool (libFuzzer would be the natural next step), a continuous-integration setup, 16-bit or float images, other border modes, images wider than 2^{31} bytes, and testing on a CPU without AVX2 (the AVX2 tests are skipped there; the scalar paths still run). The thread-count tests show equal *output* for 1 to 200 threads; they do not prove the absence of data races in a formal sense, though the design (each thread writes only its own rows and only reads the source) leaves no shared writable state to race on. Say so if asked.

12.6 How to run everything

```
make test                 # build + 24 test groups
make asan-test            # same tests under AddressSanitizer + UBSan (slower)
python3 python/make_test_images.py    # (re)generate data/*.pgm / *.ppm
python3 python/crosscheck.py          # independent NumPy/SciPy comparison
bash scripts/mutation_test.sh > results/mutation_results.txt    # 12 seeded bugs
```

Interview soundbite

"I tested at five levels: hand-computed examples, an exhaustive loop over all 16.7 million colours for gray, properties like transpose and invert symmetry for Sobel, bit-exact comparison against independent references for every edge size and thread count, and a NumPy/SciPy cross-check. Then I measured the suite itself with mutation testing: of twelve seeded bugs it initially missed one, an over-read that a scalar tail silently repaired, because my allocator padded allocations. I fixed the allocator, added an unpadded-image test and an AddressSanitizer build, and now all twelve are caught."

Chapter 13

Benchmarks: method, results, and how to read them

Sandbox numbers, not cluster numbers

Every number in this chapter was measured in the sandbox where the code was written: Intel(R) Xeon(R) Processor @ 2.10GHz, 2 visible cores, a virtual machine, g++ (Ubuntu 13.3.0-6ubuntu2 24.04.1) 13.3.0, run on 2026-10-07. They illustrate what each experiment looks like and what conclusions it supports. They are **not** bwUniCluster results and must never be quoted as such. Chapter 15 gives the scripts that produce the same tables and figures on the cluster; those numbers are yours to measure. If you have older cluster results for this project from an earlier version of the code, they came from different code and are not comparable with anything here.

13.1 Method

The benchmark program (`apps/simdip_bench.cpp`) uses one method for every configuration:

1. Allocate and fill the images *before* timing, so that page-fault cost is not measured.
2. Two warm-up calls (cache, branch predictors, OpenMP thread pool), then one call to estimate the duration and choose an iteration count so that one sample lasts at least `-min-ms` (20 ms by default).
3. Take `-reps` samples (7 to 9), each the average time per call over that many iterations. Report the **median**, plus minimum and maximum. The median is robust against an occasional disturbance by another job on a shared machine; the minimum is the best case.
4. Throughput: output pixels per second (Mpixel/s). *GB/s* is the minimum traffic (each source byte read once, each destination byte written once) divided by time, a lower bound of what the memory system moved. GFLOP/s for convolution counts $2k^2$ per pixel.
5. The program prints the CPU model, compiler, whether the build is optimised, and `OMP_PROC_BIND`/`OMP_PLACES`, so a pasted result carries its own provenance. Slurm jobs additionally write `meta.txt`.

Fairness of the comparison. The three implementations run the same image, the same kernel and the same method. "Scalar" is compiled with the auto-vectorizer switched off, "autovec" with it on, both with FMA contraction off, so that the only difference between them is the compiler's vectorization. The hand-written AVX2 path is compiled separately with `-mavx2 -mfma`. The plain `-O3` build targets baseline x86-64 (SSE2), which is what "autovec" means here; a build with `-march=native` lets the compiler use AVX2/AVX-512 and is available as an option in the cluster script (`NATIVE=1`). Quote "autovec" as "GCC `-O3` auto-vectorization, SSE2 baseline".

13.2 Main results

2048×2048	scalar (ms)	autovec (ms)	AVX2 (ms)	scalar, 2 threads	AVX2, 2 threads
gray	4.52	4.54 (1.0×)	1.16 (3.9×)	2.43 (1.9×)	0.61 (7.4×)
Sobel	8.10	2.25 (3.6×)	1.04 (7.8×)	4.14 (2.0×)	0.54 (15.0×)
conv 3x3	37.39	39.29 (1.0×)	2.08 (18.0×)	18.95 (2.0×)	1.07 (34.8×)
conv 5x5	77.72	82.75 (0.9×)	4.67 (16.6×)	39.28 (2.0×)	2.34 (33.2×)
conv 9x9	202.30	252.77 (0.8×)	13.85 (14.6×)	101.74 (2.0×)	7.02 (28.8×)
conv 11x11	304.32	370.55 (0.8×)	19.87 (15.3×)	154.70 (2.0×)	10.43 (29.2×)

Table 13.1: Median time per call, with speedup over single-thread scalar in brackets (sandbox, 2 cores).

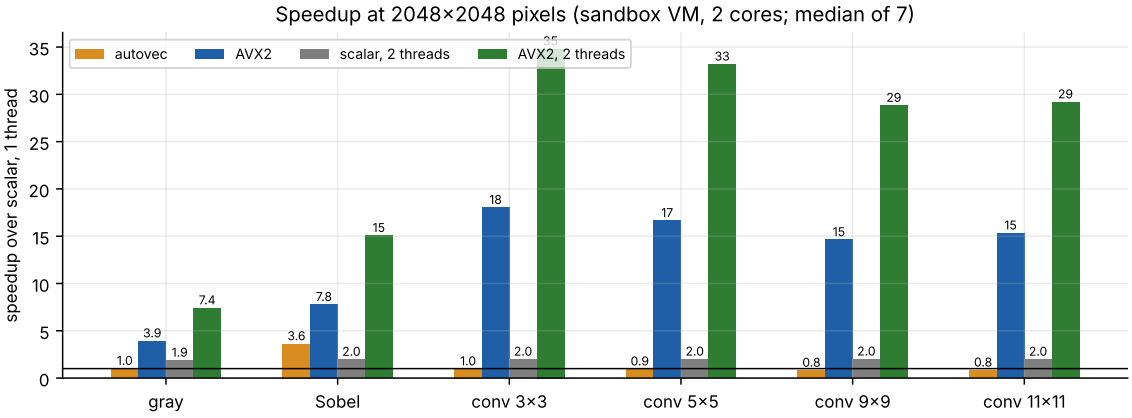


Figure 13.1: Speedups over single-thread scalar at 2048×2048.

What these data say, each point tied to a number:

- **AVX2 is the dominant factor.** On one thread it is 3.9× faster than scalar for gray, 7.8× for Sobel and 14.6× (9×9) to 18.0× (3×3) for convolution; the convolution figure is lowest for the larger kernels because it is the scalar baseline that benefits from nothing and the vector kernel that approaches its FMA ceiling. The speedup of convolution is much larger than the 8 lanes of a float vector because the scalar code needs separate multiply, add, load and index instructions per tap (and cannot be auto-vectorized, see below), whereas AVX2 retires one fused multiply-add on eight pixels per instruction.
- **GCC's auto-vectorizer helps Sobel, not gray or convolution.** It reaches 3.6× on Sobel, but 1.0× on gray (the vectorization report marks the loop as failed; the interleaved 3-byte RGB access is the likely cause but the report does not say so) and 1.0× (3 × 3) down to 0.8× (11 × 11) on convolution, i.e. no gain and for the larger kernels slightly slower than scalar (the report names the loop-nest structure, Chapter 4). Chapter 4 shows the report and which of these statements the compiler makes itself and which are interpretation.
- **OpenMP scales as the hardware allows.** With two threads scalar code gives about 2× (1.9, 2.0, 2.0), and AVX2 with two threads 7.4×, 15.0× and 34.8× over single-thread scalar: the two factors multiply. At 4096×4096 two threads give 1.91× (gray), 1.94× (Sobel) and 1.95× (convolution 9×9) over one thread, i.e. close to perfect scaling *on this machine*, because two threads do not saturate its memory (Chapter 10). That does not carry over to 32 or 64 cores: gray and Sobel will flatten once the memory system is saturated, convolution much later. The scaling script exists to measure exactly this.

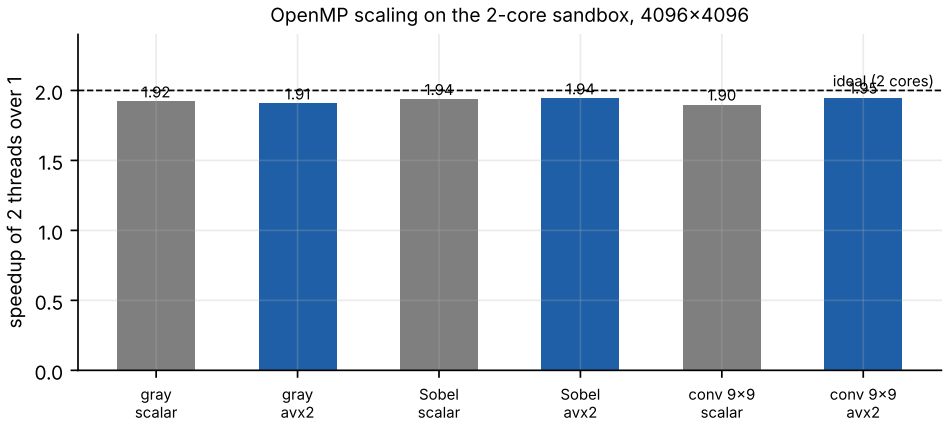


Figure 13.2: Two threads versus one at 4096×4096.

13.3 Convolution: kernel size and the ablations

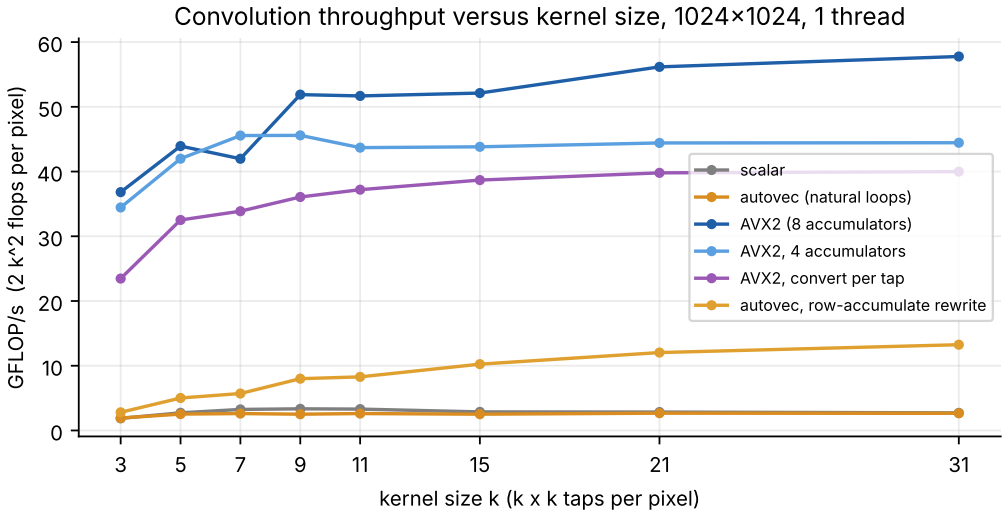


Figure 13.3: Convolution rate versus kernel size (1024×1024, one thread).

GFLOP/s, 1024^2, 1 thread	$k = 3$	$k = 9$	$k = 31$
scalar	1.9	3.3	2.7
autovec (natural loops)	1.9	2.5	2.7
AVX2 (8 accumulators)	36.8	51.9	57.8
AVX2, 4 accumulators	34.5	45.6	44.5
AVX2, convert per tap	23.5	36.1	40.0
autovec, row-accumulate rewrite	2.8	8.0	13.2

- **Throughput rises with k and then flattens at 58 GFLOP/s**, about 86 % of the nominal peak of one core (Chapter 10). For $k = 3$ the fixed per-row costs (converting a row to float, writing the output) are a larger share of the work (37 GFLOP/s); for large kernels the FMA units dominate. Scalar stays below 3.3 GFLOP/s throughout.
- **Eight accumulators beat four**: 1.14× at $k = 9$ and up to 1.30× for larger kernels: with fewer independent FMA chains the 4-cycle FMA latency is no longer hidden (Chapter 9).

- **Converting per tap costs** a factor 1.44 at $k = 9$: converting each byte to float for every tap rather than once per row spends instructions that compete with the FMAs.
- **Helping the compiler** (the row-accumulate rewrite, bit-identical to scalar) gives 8.0 GFLOP/s at $k = 9$, 3.2× the natural-loop autovec but still 6.5× below the hand-written kernel. The compiler got a long way once the loop was restructured; the remaining gap is plausibly the register blocking of the hand-written kernel (eight accumulators held in registers for a whole block of pixels across all k^2 taps); the experiment shows the gap, not its cause, and a look at the generated assembly would be needed to be sure.

Those four statements are the answer to "why not just let the compiler do it?": it is a measured, quantified answer.

13.4 Comparison with OpenCV

2048×2048, 1 thread	OpenCV (ms)	this library, AVX2 (ms)	OpenCV / ours
gray	1.75	1.18	1.5×
Sobel (3 OpenCV calls)	8.82	1.07	8.3×
conv 3x3	4.32	2.15	2.0×
conv 5x5	11.00	4.74	2.3×
conv 9x9	34.88	13.93	2.5×

This comparison is useful for orientation and dangerous as a claim. The caveats are part of the result:

- OpenCV was installed with `pip` in the sandbox (version 5.0.0, 1 thread, "optimized: True"); its build and CPU dispatch (which instruction sets it chose on this VM) were not inspected.
- Gray: OpenCV uses 14-bit coefficients, so its output differs from ours by up to one level on 7.4 % of pixels: it does more precise work.
- Sobel: our function is one fused pass producing $\min(255, |G_x| + |G_y|)$; the OpenCV figure is two `Sobel` calls in 16-bit, absolute values, a saturating add, i.e. a *pipeline* of several full-image passes and temporary images. Part of the 8.3× is fusion, not SIMD quality.
- Convolution: `filter2D` handles border modes, depths and arbitrary kernels and allocates its output; it agrees with ours to within one level on every tested kernel (identical for 3×3).
- Python call overhead is included in the OpenCV time; at 2048^2 it is negligible (microseconds against milliseconds).

The defensible sentence is: "On this machine my AVX2 kernels run 1.5× to 2.5× faster than the corresponding OpenCV calls for gray and convolution and about 8.3× for the Sobel pipeline, where most of that comes from doing it in one pass; OpenCV supports far more and I do not claim to replace it." The indefensible sentence is "faster than OpenCV".

13.5 Repeatability: how big is a "real" difference?

Two different spreads were measured, and the second one is the larger and the more instructive.

- **Within one job (E6).** The same 24 measurements repeated three times in a row differ by up to 15 % between repetitions for single-thread runs and up to 17 % for two-thread runs (the host is shared; with two threads on a 2-vCPU guest any background activity lands on a thread of the benchmark).

- **Between two passes minutes apart.** The main experiment (E1) was run twice with the identical command; the first pass is kept in `results/sandbox_example/firstrun/`. For image sizes of 1024 and above, the median difference between the two passes was 28 % for the scalar implementation (largest: 48 %), 11 % for autovec (largest 33 %) and only 2 % for AVX2 (largest 13 %). The scalar convolution was the main culprit: in the first pass it ran slower by up to about 40 % on many configurations, for a reason I cannot identify from inside a virtual machine (other tenants on the host, a different clock state, or both). All numbers in this chapter come from the second pass, and the other experiments were repeated after it for consistency.

Consequences:

- **Speedups inherit the noise of their denominator.** The AVX2 timings are stable, the scalar baseline is not, so a speedup computed from one scalar measurement can be off by tens of percent. Quote speedups as ranges or as "about", and take the baseline from several runs (the minimum of several medians is a reasonable estimate of the undisturbed scalar time).
- Differences of a few percent between two configurations, such as the stride experiment, are *not* evidence.
- Order-of-magnitude differences (the 20× of AVX2 convolution) are robust: even a 40 % error in the baseline does not change that conclusion.
- On the cluster use `-exclusive` for the cleanest numbers, repeat the jobs at different times, and quote the minimum and the spread, not just one number. Compare partitions only with the same binary and the same sizes.

13.6 What to say about performance in an interview

Interview soundbite

"On my test machine the AVX2 kernels are about 3.9 times faster than scalar for gray, 7.8 for Sobel and roughly 14.6 to 18.0 for convolution, and OpenMP multiplies that by the core count until memory bandwidth takes over. I also measured four design decisions on the convolution: eight accumulators versus four, converting once per row instead of per tap, hand-written versus a compiler-friendly rewrite, and I can show you why each matters. The AVX2 timings repeated within a couple of percent, the scalar baseline moved by tens of percent between two passes, so I quote speedups as approximate and only draw conclusions from large differences. The cluster numbers I measured with the same scripts on bwUniCluster are . . . "

(Fill the last sentence from your own cluster runs, and only from them.)

Chapter 14

Running it yourself: build, tools, experiments

Everything in this chapter works on any Linux machine with GCC (C++17, OpenMP) and an x86-64 CPU with AVX2 and FMA; the Python scripts need Python 3 with NumPy and SciPy (OpenCV is optional). On a CPU without AVX2 the library still builds; the AVX2 paths report "not supported" and the tests skip them.

14.1 Project layout

`include/simdip/`	the public API: `image.hpp`, `kernel.hpp`, `ops.hpp`
`src/`	`scalar_impl.inc` (the one scalar source, included twice), `scalar_novec.cpp`, `scalar_autovec.cpp`, `avx2.cpp`, `dispatch.cpp` (validation, OpenMP bands, impl choice), `reference.cpp`, `image.cpp`, `kernel.cpp`
`apps/`	`simdip_cli.cpp` (command line tool), `simdip_bench.cpp` (benchmark harness)
`tests/`	`test_main.cpp`: 24 test groups, no framework needed
`python/`	independent NumPy reference, cross-check, OpenCV comparison, experiment runner, figure generator
`scripts/`	`mutation_test.sh`
`slurm/`	job scripts for bwUniCluster
`results/`	saved sandbox example results, logs of tests, cross-check and mutation runs
`docs/`	the LaTeX sources of this guide

14.2 Build and test

```
make -j4                  # library, CLI, benchmark, tests -> build/
make test                 # 24 groups; the last line must read "... 0 failures"
make asan-test            # same tests with AddressSanitizer + UBSan (build_asan/)
make vec-report           # what GCC auto-vectorized -> build/vec_report_full.txt
make OPT="-O3 -march=native" BUILD=build_native    # let the compiler use the whole CPU (autovec
    only)
make OPENMP=0             # no OpenMP; the threads argument is ignored
```

The per-file compiler flags are in the Makefile and are part of the design (Chapter 4): `scalar_novec.cpp` with the vectorizer off, `scalar_autovec.cpp` with it on, both with `-ffp-contract=off`, and `avx2.cpp` with `-mavx2 -mfma`. A program that links the library calls AVX2 code only after the run-time CPU check `avx2Supported()`.

14.3 The command-line tool

```
./build/simdip info                                  # CPU features, threads
python3 python/make_test_images.py                   # writes data/*.pgm, data/*.ppm
./build/simdip gray  data/astronaut.ppm  out_gray.pgm --impl avx2 --threads 2
./build/simdip sobel data/camera.pgm     out_sobel.pgm --impl avx2
```

```
./build/simdip conv  data/camera.pgm     out_blur.pgm --kernel gauss --size 9 --sigma 2.5 --impl
    avx2
./build/simdip conv  data/camera.pgm     out_sharp.pgm --kernel sharpen --impl scalar
```

Images are read and written as PNM (PGM for gray, PPM for colour). View them with any image viewer that supports PNM, or convert them with OpenCV or Pillow (for example `cv2.imwrite` on the result of `cv2.imread`). Running the same command with `-impl scalar` and `-impl avx2` and comparing the files with `cmp` is the quickest sanity check: for gray and Sobel they must be identical.

14.4 The benchmark tool

```
./build/simdip_bench --op gray,sobel,conv --impl scalar,autovec,avx2 \
    --sizes 512,1024,2048 --ksizes 3,5,9 --threads 1,2 --reps 9 --csv out.csv --label try1
./build/simdip_bench --op conv --impl avx2 --ablation --sizes 1024 --ksizes 3,9,31 --csv abl.csv
./build/simdip_bench --op copy --sizes 4096 --threads 1,2       # memory-bandwidth yardstick
./build/simdip_bench --op sobel,conv --sizes 4096 --stride 4160 --ksizes 9    # explicit row stride
```

Options: `-op` (gray, sobel, conv, copy), `-impl`, `-sizes` (image side in pixels), `-ksizes`, `-threads`, `-reps`, `-min-ms`, `-stride` (bytes), `-aspect N` (height = width/N, for non-square images), `-ablation`, `-csv`, `-label`. The CSV has one row per measurement with the columns `label, op, impl, threads, width, height, stride, ksize, iters, reps, median_ms, min_ms, max_ms, mpix_s, gb_s, gflops`. Build with `-O0` and the program prints a warning that the timings are meaningless.

14.5 Python scripts

`make_test_images.py`	generates the test pictures (a photograph, colour images, noise, a checkerboard) into `data/`
`ref_numpy.py`	the independent reference implementation (NumPy + SciPy)
`crosscheck.py`	runs the CLI for all implementations and thread counts, compares with the reference; prints the number of comparisons and mismatches
`compare_opencv.py`	agreement and speed comparison against OpenCV (needs `cv2`); `-bench-csv`, `-size`, `-out`
`run_experiments.py`	reruns the experiments E1–E5 into `results/`
`make_figures.py`	produces every figure, table and number macro used in this PDF from the CSV files
`extract_code.py`	cuts the code listings of this guide out of the source files (so listings cannot drift from the code)

14.6 Experiments

id	file	question
E1	`E1_main.csv`	every op × implementation × size × kernel, 1 and 2 threads
E2	`E2_sizes.csv`	throughput versus image size (cache effects)
E3	`E3_stride_*.csv`	row stride equal to a page multiple versus padded by 64 bytes
E4	`E4_ksize.csv`	convolution versus kernel size, with the three ablations
E5	`E5_copy.csv`	the memory-bandwidth yardstick (`memcpy`)
E6	`E6_repeat_*.csv`	the same measurement three times: repeatability
E7	`E7_scaling.csv`	(cluster) OpenMP scaling 1 to 32 threads; not available in the sandbox (2 cores)

14.7 Using the library from your own code

```
#include "simdip/ops.hpp"
using namespace simdip;

Image rgb = readPNM("in.ppm");                    // 3 channels
Image gray(rgb.width(), rgb.height(), 1);         // destination: right size, 1 channel
rgbToGray(rgb, gray, {Impl::Avx2, 4});            // AVX2, 4 OpenMP threads
Image edges(gray.width(), gray.height(), 1);
sobel(gray, edges, {Impl::Avx2, 4});
Image blurred(gray.width(), gray.height(), 1);
convolve(gray, blurred, kernels::gaussian(9, 2.5), {Impl::Avx2, 4});
```

The functions throw `std::invalid_argument` for wrong sizes, overlapping buffers or an unusable kernel; they never read or write outside the pixels of a row.

Chapter 15

Running on bwUniCluster 3.0: step by step

bwUniCluster 3.0 is the shared HPC system operated by KIT for universities in Baden-Württemberg, including Freiburg. You use it by logging in to a *login node*, preparing your work there, and handing the real computation to a job scheduler called **Slurm**, which runs it on a *compute node*.

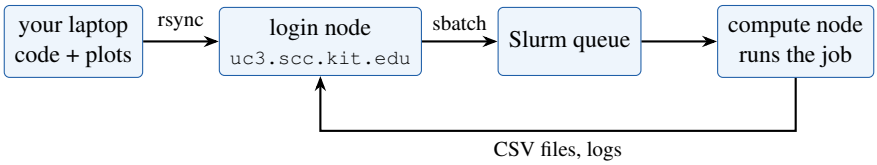
In plain words

Mental model. The login node is a shared reception desk: fine for editing files and submitting forms, *not* for computing (heavy computations on login nodes are not allowed). The compute nodes are the workshops. A *job script* is the form you hand in: "give me 1 core for 30 minutes on an Intel node, then run these commands". Slurm queues the form and runs it when a node is free.

Facts and where they come from

Queue names, limits and hardware below come from the official bwHPC wiki pages listed at the end of this chapter, which I read while preparing this guide. HPC sites change these things. Before relying on a number, confirm it on the cluster with `sinfo -s` (queues), `module avail` (software) and the wiki.

15.1 The workflow in one picture



15.2 One-time setup: account and login

1. **Registration.** The login documentation lists three prerequisites: complete the 3-step registration procedure, set a *service password* for bwUniCluster 3.0, and set up a *second factor* (a TOTP app on your phone). You enter the one-time code first and then the service password when you log in. If you have used the cluster before, this is probably done; if the login fails, this is the first thing to re-check.
2. **Network.** Access is limited to the BelWü research network. On the Freiburg campus network you are normally inside it; from home use the university VPN or an SSH jump host. A connection that times out from outside is not a bug in your command.
3. **Log in.**

   ```
   ssh <your_username>@uc3.scc.kit.edu
   ```

   Your username is a university prefix plus your local name (for KIT users, for example, `ka_ab1234`); the registration pages show yours. Alternative host names listed on the wiki: `bwunicluster.scc.kit.edu`, `uc3-login1.scc.kit.edu`, `uc3-login2.scc.kit.edu`.

15.3 Where to put files

`$HOME`	Permanent, backed up, same on all nodes. Put source code and job scripts here. Our project is a few megabytes, so it lives here.
Workspaces	For large temporary data (restart files, big outputs). Created with `ws_allocate <name> <days>`, found with `ws_find <name>`, listed with `ws_list`, extended with `ws_extend <name> <days>`; they expire. You do not need one for this project unless you benchmark with many gigabytes of images.
`$TMPDIR`	Fast node-local disk, exists only while a job runs. Irrelevant here.

15.4 Step 1: copy the project to the cluster

From your laptop (in the folder that contains `simd_imgproc/`):

```
rsync -av --exclude 'build*' --exclude results/sandbox_example simd_imgproc/ <user>@uc3.scc.kit.edu
    :~/simd_imgproc/
```

(Or push to a private GitHub repository and `git clone` on the cluster.) Then on the cluster:

```
cd ~/simd_imgproc
mkdir -p logs results data
```

The C++ benchmark uses synthetic images and needs no data files. The cross-check needs the test images; `data/` is included in the package already, so the rsync above copies it.

15.5 Step 2: look around

```
module avail compiler/gnu        # which GNU compiler versions exist
module spider gnu                # (Lmod) also finds modules with unmet dependencies
sinfo -s                         # summary of queues (partitions) and node counts
squeue --me                      # your own jobs
lscpu | grep -E 'Model name|Flags' | head   # does the node you land on list avx2 and fma?
```

The cluster uses **Lmod**: `module load`, `module list`, `module purge` and `module spider` are the commands you need. The wiki lists GNU compilers `compiler/gnu/11.4` and `compiler/gnu/14.2` (`13.3` is also referenced). `slurm/env.sh` tries `compiler/gnu/14.2` first and falls back to the default `compiler/gnu`; edit it if `module avail` shows different versions. The project needs C++17 and OpenMP. It was built and tested with GCC 13.3; any recent GCC should work, but older ones were not tried.

15.6 The queues you will use

queue	node type	cores / memory	max wall time	use it for
`dev_cpu`	development queue	as `cpu`	30 min	compile, test, short checks
`cpu_il`	Intel Xeon Platinum 8358 (Ice Lake)	64 cores, 256 GiB	72 h	benchmark
`cpu`	AMD EPYC 9454	96 cores, 384 GiB	72 h	benchmark
`highmem`	AMD EPYC 9454	96 cores, 2.3 TiB	72 h	not needed

Development queues (`dev_` prefix) have a 30-minute limit, allow only *one running job at a time* and at most 3 queued, and are meant for "developing, compiling and testing code". The wiki says not to misuse

them for regular short-running production jobs. So: build and test in `dev_cpu`; run the benchmarks in `cpu_il` and `cpu`.

Both CPUs have AVX2 and FMA; the sandbox numbers do not transfer

The library's AVX2 path needs AVX2 and FMA; the two benchmark partitions use CPUs of generations that provide both, but confirm it on the node with `lscpu` (the job's `meta.txt` records the CPU model). The two partitions differ in core count, caches, memory bandwidth and clock, so you will get *two different tables*, and both differ from the sandbox. That is the point of running both.

15.7 Step 3: build and test on a compute node

```
sbatch slurm/01_build_test.sbatch
squeue --me                          # PD = pending, R = running. Check again after a minute or two.
cat logs/simdip-build_<jobid>.out # expect "... 0 failures" twice (normal build and sanitizer build)
     and "build + tests OK"
```

Be kind to the scheduler

The wiki asks you not to run `squeue` or other Slurm commands in loops or with `watch`, and to leave at least 30 seconds between queries. Submit, wait, check.

The job runs the unit tests, then the AddressSanitizer build of the same tests, then the NumPy cross-check if `python3` has NumPy and SciPy (otherwise it prints a message and skips it, which is not a failure). The test log is saved as `results/test_log_cluster.txt`. Interactive alternative (from the wiki):

```
salloc -p dev_cpu -n 1 -t 20 --mem=4000      # 1 task, 20 minutes, 4000 MB; you land on the node
module load compiler/gnu && make && make test
exit                                          # give the node back!
```

15.8 Step 4: the benchmark jobs

```
bash slurm/submit_all.sh          # 2 jobs per partition: single-core (02) and OpenMP scaling (03)
squeue --me
```

script	resources	what it measures
`02_bench_single.sbatch`	2 cores, 12 GB, 1.5 h	the same experiment set as the sandbox example: E1 (all ops, implementations, sizes 256–4096, kernels 3–11, 1 and 2 threads), E2 (size sweep), E3 (page-multiple stride versus +64 B), E4 (kernel sizes up to 31 with ablations), E5 (`memcpy` yardstick), E6 (repeatability), plus the OpenCV comparison if `cv2` is available. Results in `results/<partition>/`.
`03_bench_scaling.sbatch`	32 cores, 24 GB, 1 h	E7: threads 1, 2, 4, 8, 16, 32 for scalar and AVX2, sizes 2048, 4096, 8192; also the `memcpy` yardstick with the same thread counts. Threads pinned with `OMP_PROC_BIND=close`, `OMP_PLACES=cores`.
`04_profile.sbatch`	2 cores, `dev_cpu`	compiler vectorization report and a gprof flat profile of the scalar and autovec code

15.8.1 What the #SBATCH lines mean

`-partition=cpu_il`	Which queue (node type). Overridden by `sbatch -partition=cpu ...`.
`-nodes=1 -ntasks=1`	One node, one process.
`-cpus-per-task=1` (or 32)	How many cores the process may use. The benchmark job asks for two cores because it also times 2 threads; its single-thread numbers use one of them; the scaling job asks for 32 and sets `OMP_NUM_THREADS` from it.
`-mem=12000mb`	Memory. The largest image here (8192^2) needs well under 1 GB; ask explicitly because the default per core is small.
`-time=01:00:00`	Wall-time limit. The job is killed at the limit.
`-output=logs/%x_%j.out`	Log file: `%x` = job name, `%j` = job number. **The directory must already exist** (`mkdir -p logs`) or the job fails immediately.
`srun -cpu-bind=cores`	Starts the program on the allocated core and pins it so it cannot migrate during the measurement. The scaling job uses `-cpu-bind=none` and lets OpenMP do the pinning.

15.8.2 Benchmark hygiene (what makes a number trustworthy)

- **Warm-up, repetitions, median** are built into `simdip_bench`; the spread is in the CSV (`min_ms`, `max_ms`).
- **Same binary, same input, same settings** on both partitions; only the node changes.
- **The node may be shared.** Other jobs on the same node disturb timings, in particular memory bandwidth, which is what the multi-thread results measure. For the scaling job the clean version is `sbatch -exclusive -partition=cpu_il slurm/03_bench_scaling.sbatch` (longer queue wait). A scaling curve measured on a busy node says more about the neighbours than about your code.
- **Repeat.** Run each job at least twice at different times and compare. The E6 block inside the job measures the spread within one job; the spread *between* jobs can be larger (in the sandbox the scalar baseline differed by a median of 28 % and up to 48 % between two passes of the same command, Chapter 13).
- **Report what you measured**: "median of 9 samples, single core, GCC ..., `-O3`, partition ..., node ...".
- **Optional extra:** `NATIVE=1 sbatch -partition=cpu_il slurm/02_bench_single.sbatch` rebuilds with `-march=native` on the compute node into a separate build directory and results folder `results/<partition>_native/`. The "autovec" column may then use AVX2 or AVX-512 and FMA contraction is still off for that file. Compare with the portable build. A native binary can fail with "illegal instruction" on a different CPU type, so it is named per partition.

15.9 Step 5: profile

```
sbatch slurm/04_profile.sbatch       # dev_cpu, about 2 minutes
less results/vec_report_cluster.txt  # which loops did this GCC vectorize?
less results/gprof_flat.txt          # function-level table
```

The vectorization report is the cluster compiler's answer to Chapter 4: a newer or older GCC can make different decisions, and that is a legitimate thing to report.

15.10 Step 6: bring the results home

```
rsync -av <user>@uc3.scc.kit.edu:~/simd_imgproc/results/ ./cluster_results/
```

Then turn them into the same figures and tables as in this guide, without touching the guide's own files:

```
python3 python/make_figures.py --results cluster_results/cpu_il --out cluster_results/cpu_il/report
    \
      --testlog cluster_results/test_log_cluster.txt --crosslog cluster_results/
    crosscheck_log_cluster.txt
```

This writes `report/figures/*.pdf` (speedups, size sweep, kernel sizes, bandwidth, stride, and `f_scaling.pdf` from the scaling job) and `report/snippets/*.tex` (tables and number macros). It needs Python with NumPy, SciPy and matplotlib on your laptop; the `-testlog`/`-crosslog` options can be left out if you only want the benchmark tables. Where the guide's text quotes a sandbox number, read your own value in the corresponding table instead; the chapters were written around the structure of the results, not around a specific machine.

15.11 When something goes wrong

Symptom	Likely cause and fix
`ssh` hangs or is refused	Not in the BelWü network: connect to the university VPN. Wrong code order: one-time code first, then service password.
`module: command not found` in a script	The script is not run through a login shell. `slurm/env.sh` checks for it; check `module list` at the top of the log.
Module not found	Version differs: `module avail compiler/gnu`, `module spider gnu`, then edit `slurm/env.sh`.
Job stays `PD` (pending)	Normal. `squeue -me` shows a reason: `Priority`/`Resources` = waiting for a free node; for `dev_cpu` you may only have one running job.
Job fails in 1 second, empty log	`logs/` directory missing, or a typo in `-partition`. Look at `scontrol show job <id>` and run `sinfo -s`.
`Killed` / out-of-memory	Raise `-mem`.
Benchmark prints "avx2+fma: no"	The node's CPU lacks AVX2/FMA or the job landed somewhere unexpected; check `meta.txt`. AVX2 rows are skipped automatically.
"Illegal instruction"	A `-march=native` binary ran on a different CPU type; rebuild on the node that runs it.
Scaling curve flat after a few threads	Expected for gray and Sobel when memory bandwidth saturates; check against the `memcpy` scaling in `E7_copy_scaling.csv`. Also check the `OMP_PROC_BIND` line printed at the top of the output.
Numbers differ a lot between runs	Shared node. Use `-exclusive`, more repetitions, or compare medians from separate runs.
Tests fail on the cluster but not locally	Should not happen; the results are deterministic. Check `g++ -version` and that the sources are current, then investigate: it would be a real finding.

15.12 How to describe it truthfully

After you have run the jobs, a sentence you can use, with the brackets filled from *your* CSV files:

> "I benchmarked the library on bwUniCluster 3.0 nodes (Intel Xeon Platinum 8358 and AMD EPYC 9454) with GCC and `-O3`: single core, median of 9 samples, AVX2 versus scalar speedups of [...] for gray, [...] for Sobel and [...] for 5×5 convolution, and OpenMP

> scaling to [...] threads, where gray and Sobel flattened at [...] threads because of memory bandwidth."

Only claim what the logs and `meta.txt` show. If something did not run, say so. If you have older cluster numbers from a previous version of this project, do not mix them with these.

15.13 Sources

All retrieved while preparing the project-2 guide and reused here unchanged; re-check them, since clusters change.

- Batch queues: `https://wiki.bwhpc.de/e/BwUniCluster3.0/Batch_Queues`
- Running jobs (`sbatch`, `salloc`, monitoring): `https://wiki.bwhpc.de/e/BwUniCluster3.0/Running_Jobs`
- Software modules (Lmod): `https://wiki.bwhpc.de/e/BwUniCluster3.0/Software_Modules`
- Login, registration, network restriction: `https://wiki.bwhpc.de/e/BwUniCluster3.0/Login`
- Hardware and file systems: `https://wiki.bwhpc.de/e/BwUniCluster3.0/Hardware_and_Architecture`
- Workspaces: `https://wiki.bwhpc.de/e/Workspaces`

Chapter 16

The resume bullets, one by one

Wording

The bullets below are paraphrased from the project description ("optimized grayscale conversion, Sobel edge detection and 2D convolution; scalar, OpenMP and AVX2 implementations; vectorized operations and cache-aware memory access; benchmarks of runtime, throughput and speedup across resolutions and kernel sizes; automated correctness tests against a scalar reference"). Replace the quoted lines with your exact resume text; the explanations stay valid. Anything in square brackets is a number you must take from your own cluster runs.

For each bullet: what it means in plain words, how it was done, where to find it, what evidence exists, and the follow-up question to expect.

16.1 Bullet 1: three operations, three implementations

"Implemented grayscale conversion, Sobel edge detection and 2D convolution in C++ with scalar, OpenMP-parallel and AVX2-vectorized versions."

What Three image operations, each in three implementation styles (scalar, compiler-vectorized, hand-written AVX2), each runnable on 1 to n OpenMP threads: $3 \times 3\times$ threads, all producing the same pixels.

How One scalar source compiled two ways; AVX2 intrinsics in one translation unit with run-time CPU check; row-band parallelism with `omp parallel` over contiguous bands (Chapters 4–9).

Where `src/scalar_impl.inc`, `src/avx2.cpp`, `src/dispatch.cpp`.

Evidence 24 test groups; AVX2 results equal an independent `fmaf` reference bit for bit; gray verified on all 2^{24} colours.

Expect "Walk me through the AVX2 grayscale." (3 loads of 16 bytes, 9 `pshufb` to deinterleave R, G, B, widen to 16 bits, multiply by 77/150/29, add 128, shift right 8, pack; Chapter 7.) "Why not float for gray?" (Integer keeps 16 pixels per register and exactness; 8-bit coefficients cost 0.2 % coefficient error.)

16.2 Bullet 2: vectorization and cache-aware access

"Optimized for cache and SIMD: vectorized arithmetic, aligned rows, a ring of converted rows for convolution, register blocking with independent accumulators."

What The arithmetic is data-parallel across pixels, and the data re-used by neighbouring outputs is kept in cache or registers.

How 64-byte-aligned rows; stride that avoids exact multiples of 4096 bytes; convolution converts each source row to float *once* into a ring of k padded rows (the working set is $4k(w+2r)$

bytes, in L2); eight independent accumulators to hide FMA latency; one broadcast weight per tap.

Where `convolveRowsImpl<8>` in `src/avx2.cpp`; Chapters 9 and 10.

Evidence Ablations on the convolution (Chapter 13): 8 accumulators are 1.14× faster than 4 at $k = 9$; converting per tap costs 1.44×; the compiler-friendly rewrite reaches only about 1/6.5 of the hand-written kernel's rate. Stride experiment: effect within noise on the sandbox.

Expect "Does the stride padding actually help?" (Say what you measured: on the VM, within noise; the theoretical failure mode is real, and you repeat the test on the cluster.) "How close to peak?" (About 86 % of the nominal FMA peak on the sandbox core; give your cluster figure.)

16.3 Bullet 3: benchmarking across resolutions and kernel sizes

"Benchmarked runtime, throughput and speedup across image resolutions and kernel sizes."

What A harness with warm-up, calibrated iteration counts, median of repetitions, CSV output, and a memory-bandwidth yardstick.

How `apps/simdip_bench.cpp`; experiments E1–E7 (Chapter 14); Slurm jobs for two CPU types; figures from `python/make_figures.py`.

Where Chapters 13 and 15.

Key numbers to prepare (yours)
AVX2 over scalar, single core: gray [ ], Sobel [ ], convolution 3×3 [ ] and 9×9 [ ], on `cpu_il` and on `cpu`; OpenMP scaling to [ ] threads and the thread count at which gray and Sobel stop scaling; memcpy bandwidth.

Expect "Why does gray stop scaling before convolution does?" (Memory bound versus compute bound; show the roofline argument and the memcpy scaling curve.) "How reproducible are your numbers?" (State the spread you measured, Chapter 13.)

16.4 Bullet 4: correctness testing

"Developed automated correctness tests comparing all implementations against a scalar reference."

What A layered test suite whose job is to catch the bugs specific to SIMD code.

How Edge-size matrix around vector widths, padded and unpadded images, 1 to 200 threads, bit-exact comparisons, exhaustive gray, property tests, AddressSanitizer, an independent NumPy/SciPy cross-check, and a mutation test of the suite itself (12 of 12 seeded bugs caught; Chapter 12).

Evidence 4 269 710 checks, 0 failures; 786 cross-check comparisons, 0 mismatches.

Expect "What bug did the tests find?" (The over-read in the vector loop that the scalar tail masked; found by the mutation test because the allocator hid it.) "How do you test floating-point code?" (Fixed accumulation order, an FMA reference for bit-exactness, exact kernels for zero tolerance, a proven error bound for the rest.)

16.5 Bullet 5: Python tooling

"Python for reference implementation, cross-validation, experiment automation and plotting."

What An independent implementation of the specification, tools that drive the C++ binaries, and the figure generator.

Where `python/ref_numpy.py`, `crosscheck.py`, `compare_opencv.py`, `run_experiments.py`, `make_figures.py`.

Expect "Why a second implementation in Python?" (It shares no code and no misunderstanding with the C++ reference; it catches specification errors that self-consistency cannot.)

16.6 Wording to consider once your cluster numbers exist

Candidate bullets that survive a skeptical read, each fact checkable in the project:

- *Built a C++17 image-processing library (RGB-to-gray, Sobel, $k \times k$ convolution) with scalar, auto-vectorized and hand-written AVX2 paths plus OpenMP row-band parallelism; AVX2 output is bit-identical to an independent FMA reference and gray is verified on all 2^{24} inputs.*
- *Designed an AVX2 convolution around a ring of float-converted rows (L2-resident), eight independent FMA accumulators and run-time CPU dispatch; reached [X] % of single-core FMA peak and [Y]$\times$ over scalar on bwUniCluster (Xeon 8358), [Z]$\times$ on EPYC 9454.*
- *Quantified optimization choices with ablations (accumulator count, convert-once, compiler rewrite) and a roofline analysis using a measured memcpy yardstick; gray/Sobel saturate memory bandwidth at [N] threads, convolution scales to [M].*
- *Verified with 4 269 710 checks, AddressSanitizer, a NumPy/SciPy cross-check and mutation testing (12/12 seeded bugs detected).*

Remove any bracketed claim you did not measure. Do not write "faster than OpenCV" (Chapter 13), do not write a speedup that is not from your cluster run, and do not write "cache-optimized" unless you can describe, in one sentence, what is in which cache.

Chapter 17

Interview questions and answers

Answers are written to be said aloud in 20–60 seconds. Numbers that depend on the machine are marked *[yours]*: use your cluster value, or say "on my test machine" if you quote the sandbox figure.

17.1 Orientation

Q. Describe the project in thirty seconds.

A C++ image-processing library with three operations, RGB-to-gray, Sobel edge strength and $k \times k$ convolution, each in a scalar version, a compiler-vectorized version and a hand-written AVX2 version, and each parallelizable over rows with OpenMP. I wrote a test suite that checks all combinations against a reference, including bit-exact checks for the floating-point convolution, and a benchmark harness whose results I measured on bwUniCluster. The point was to understand what the hardware does with these kernels and to be able to explain each speedup.

Q. What is the single most important result?

That the three kernels sit at different places on the roofline. Gray and Sobel stream pixels and end up limited by memory bandwidth after vectorization; convolution does $2k^2$ flops per pixel and is limited by the FMA units. So the extra work of intrinsics paid off most for convolution *[yours: AVX2 over scalar single core]*, and OpenMP scaling flattens for gray and Sobel when bandwidth saturates *[yours: thread count]*.

Q. What would you do differently or next?

Row blocking in the convolution (compute two or four output rows from each loaded float row to cut loads), separable-kernel detection (a $k \times k$ separable kernel is $2k$ taps instead of k^2), non-temporal stores for large images, an AVX-512 path with a run-time dispatch, first-touch parallel initialisation for NUMA, and a GPU port of the same three kernels for comparison.

17.2 SIMD and AVX2

Q. What is SIMD, and what is the vector width here?

One instruction operating on several data elements. An AVX2 register is 256 bits: 32 bytes, 16 16-bit integers or 8 floats. Gray and Sobel process 16 pixels per iteration in 16-bit lanes; the convolution processes 8 pixels per FMA, 64 per block of 8 accumulators.

Q. Why 16-bit lanes for gray and Sobel?

The intermediate values fit: the gray sum is at most 65 408 (unsigned 16-bit); Sobel magnitudes at most 2040 (signed 16-bit). Using 32-bit lanes would halve the pixels per instruction for no benefit.

Q. AVX2 `packus` gives scrambled pixel order. Why, and how do you fix it?

AVX2 integer instructions mostly work independently in each 128-bit half. `packus` packs the low halves of both inputs into the low half of the result and the high halves into the high half, so the 64-bit quarters come

out in the order 0, 2, 1, 3. A `permute4x64` with the right selector (or `permutevar8x32` after a 32-bit pack) restores the order.

Q. How do you deinterleave RGB in the grayscale kernel?

Three 16-byte loads cover 16 pixels (48 bytes). Each of R, G, B is spread over all three registers in a stride-3 pattern, so for each colour I use one `pshufb` per register with a constant mask that picks the right bytes (and zeros the others with index `0x80`), then OR the three results. 9 shuffles and 6 ORs. The masks are compile-time constants.

Q. Why do the loads happen at `x-1`, `x` and `x+1` in Sobel rather than shuffling?

Unaligned loads on current cores are cheap when they do not cross a cache line, and three loads per row avoids lane-crossing shuffles. The price is some redundant cache traffic, which stays in L1.

Q. Aligned versus unaligned loads: does it matter?

On recent Intel and AMD cores an unaligned load that stays within a cache line costs the same as an aligned one; one that splits a line costs more. I align the rows to 64 bytes so that row starts are line-aligned, but kernels read at arbitrary x, so I use unaligned instructions throughout.

Q. What happens on a CPU without AVX2?

The library checks `__builtin_cpu_supports` at run time and throws if AVX2 is requested but not available; scalar and autovec still work. The AVX2 file is the only one compiled with `-mavx2 -mfma`, so no AVX2 instruction can leak into code that runs before the check.

Q. Why not `-march=native` for everything?

The binary then only runs on that CPU type (an "illegal instruction" on another cluster partition), and the compiler may fuse multiply-adds in the "scalar" code, silently changing numerical results. I keep the three implementations' flags explicit and test them separately.

Q. What did the auto-vectorizer do?

GCC vectorized the Sobel loop with 16-byte (SSE2) vectors, did not vectorize gray or the natural convolution loop nest. For the convolution the report names the loop-nest structure. When I rewrote the loop to accumulate whole rows, GCC vectorized it with identical output bits and got to about 1/6.5 of the AVX2 kernel. So: it helps, it needs coaxing, and it did not match hand-written register blocking.

17.3 Convolution

Q. Explain the convolution kernel's structure.

For each output row: convert the one new source row to float into a ring of k padded rows (border replicated, so no border logic in the inner loop); then for blocks of 64 output pixels hold 8 float accumulators in registers; for each of the k^2 taps broadcast the weight and do 8 FMAs reading from the padded rows; finally convert to int with rounding, pack with saturation, fix the lane order, store.

Q. Why eight accumulators?

FMA latency is about four cycles and a core can start two per cycle, so at least eight independent chains are needed to keep both units busy. I measured it: 4 accumulators are $1.14\times$ slower at $k = 9$ on my sandbox core.

Q. Why convert to float once per row?

A source pixel is used k^2 times. Converting for every tap spends instructions that compete with the FMAs; the per-tap variant measured $1.44\times$ slower at $k = 9$.

Q. What stops you from reaching the FMA peak?

I did not profile it down to the cause, so this is a hypothesis: per tap the kernel issues 8 FMAs but 9 loads (8 vectors from the ring plus the broadcast weight), and a core with two load ports and two FMA ports would then be load-bound at about 8/9 of the FMA peak, plus the overhead of the row conversion and stores. On the sandbox core I reached about 86 % of the nominal peak *[yours]*, which is consistent with that but does not prove it. The way to test it is hardware counters (`perf stat`) or a variant with fewer loads, such as blocking over two output rows, where each loaded vector feeds two sets of accumulators.

Q. How do you handle the border?

Replicate the edge pixel. In the convolution, the padded float row has r extra entries on each side so there is no condition in the inner loop. In y the ring is indexed by clamped source row, so row 0 can appear several times. In Sobel the first and last columns go through a scalar helper.

Q. Why is the result bit-identical to your reference but not to scalar?

Because the AVX2 code uses fused multiply-adds (one rounding) and the scalar code does a multiply and an add (two roundings). The reference uses `fmaf` in the same order, so it matches AVX2 exactly. Scalar differs from it by at most one grey level, only on rare near-ties. For binomial kernels everything is exact and all agree with zero tolerance.

Q. What about a separable kernel?

A separable $k \times k$ kernel (Gaussian, box) is an outer product of two vectors, so it can be applied as a horizontal then a vertical pass: $2k$ taps instead of k^2. My library does not exploit this; it treats the kernel as general. With a 9×9 Gaussian, the separable version would do about $4.5\times$ less arithmetic but needs a temporary image, which changes the memory behaviour. I would implement it as the next step.

17.4 OpenMP and memory

Q. How is the work divided?

The image is cut into n contiguous row bands. Each thread writes only its own rows and reads the source, so there are no data races and the output is independent of the thread count; the tests check this for 1 to 200 threads.

Q. Why does gray stop scaling?

It reads 3 bytes and writes 1 per pixel with very little arithmetic, so after vectorization the memory bandwidth limits it. One core cannot saturate the memory controllers, so scaling is good up to a point and then flat. My memcpy yardstick shows the ceiling *[yours: threads at which memcpy flattens]*.

Q. What is false sharing, and does it occur here?

Two threads writing different variables in the same cache line force that line to bounce between cores. Here threads write different rows of the output; only at band boundaries could two threads touch the same line, and with 64-byte-aligned rows that does not happen unless the stride is not a multiple of 64.

Q. What is NUMA, and why do you pin threads?

On a multi-socket node each socket has its own memory; accessing the other socket's memory is slower. `OMP_PROC_BIND=close` and `OMP_PLACES=cores` keep threads on distinct physical cores near each other.

A fuller solution initialises the image in parallel with the same band layout so each page is local to the thread that uses it (first touch). I did not implement that.

Q. Why avoid a stride of exactly 4096 bytes?

Addresses a multiple of 4096 apart map to the same L1 set (4K aliasing; also associativity conflicts in L2), and Sobel reads three rows and writes one at the same column. In my measurements on the sandbox the effect was inside run-to-run noise; on the cluster *[yours]*. The padding is cheap (1.6 %) insurance against a failure mode the theory predicts.

17.5 Numerics and testing

Q. How do you know the AVX2 grayscale is correct?

It is checked for all 16 777 216 colours against the reference, plus edge sizes, strides and thread counts. That is exhaustive for the arithmetic; the remaining risk is the data movement at row ends, which the edge-size and no-padding tests with AddressSanitizer cover.

Q. What is the error of your float32 convolution?

A bound of $\gamma_n \cdot 255 \sum |w|$ with $n = k^2$; the largest measured error was 0.092 of that bound. In output terms an error matters only when the exact sum is within about 10^{-4} of a rounding boundary, and then the result changes by one grey level.

Q. Why round-half-even?

It is IEEE's default, what `cvtps_epi32` and `lrintf` do, and it avoids a statistical bias that round-half-up introduces. It also makes scalar and vector code agree on exact ties.

Q. What is mutation testing and what did it find?

Seed a deliberate bug, run the tests, and see whether the suite fails. Of twelve mutants, one initially survived: a vector over-read that the scalar tail then repaired. My allocator rounded sizes up and hid the over-read from AddressSanitizer. I made allocation exact, added unpadded-image tests, and now all twelve are caught.

17.6 Benchmarks

Q. How do you benchmark?

Warm up twice, calibrate the number of iterations so one sample lasts at least 20 ms, take 7 to 9 samples and report the median with min and max; images allocated and touched before timing; thread placement pinned; compiler, CPU and `OMP_*` variables printed with the results. Single-thread numbers use one core only.

Q. Why median rather than mean or minimum?

On a shared machine an occasional disturbance inflates a sample; the mean follows it, the median does not. I also keep the minimum as the best case. For a deterministic kernel the minimum is a good estimator of the undisturbed time, the median is a more honest typical value.

Q. How reproducible were your numbers?

On the sandbox VM the AVX2 timings repeated within a few percent, but the scalar baseline differed by a median of 28 % (up to 48 %) between two passes of the same command, so I report speedups as approximate and only interpret large differences. On the cluster *[yours: spread between repeated jobs]*.

Q. Did you compare against OpenCV?

Yes, single-threaded, on one machine. AVX2 gray and convolution were 1.5–2.5× faster than the corresponding OpenCV calls, Sobel 8.3×, but that last figure compares a fused single pass with a multi-call OpenCV pipeline, and OpenCV's gray uses more precise 14-bit coefficients. OpenCV supports many more types and border modes. I treat it as a sanity check, not a claim.

17.7 Connecting to GPU-oriented image and data-processing libraries

Careful

This project is CPU-only. I have not implemented it on a GPU, and you should say so. What you *can* do is show that you understand how the same three kernels map to a GPU, which is relevant for a team that builds GPU image and data-processing libraries. The library names below (NPP, nvJPEG, DALI, CV-CUDA) are NVIDIA products as far as I know; confirm what the team on the job posting actually works on before quoting any of it.

Q. How does your CPU design map to a GPU?

SIMD lanes correspond to threads of a warp executing in lock-step (SIMT); a block of 64 pixels in registers corresponds to a thread computing several outputs; the ring of float rows in L2 corresponds to a tile with halo in shared memory; contiguous 16-byte loads correspond to coalesced global-memory access; the roofline analysis is identical, with a much higher compute ceiling and a bandwidth ceiling that matters even more.

Q. How would you write the convolution on a GPU?

Each thread block loads a tile of the source plus a halo of r pixels into shared memory (border replicated while loading), synchronises, then each thread computes one or a few output pixels from shared memory with the weights in constant memory or registers. Choices: tile size versus occupancy, bank conflicts in shared memory, using 4 pixels per thread with vector loads. For large k and separable kernels, two passes. I would measure against the bandwidth of a copy kernel, exactly as with `memcpy` here.

Q. What would you test differently on a GPU?

The same layers: reference equivalence over edge sizes (block and tile boundaries are the analogue of vector tails), bit-exactness where the arithmetic is integer, an FMA-ordered reference for float, tests for different strides and pitched allocations, and a race and out-of-bounds checker (the analogue of AddressSanitizer is `compute-sanitizer`).

Q. Do you know CUDA?

Answer truthfully. If you have not written kernels, say what you do know (the execution model, memory hierarchy, coalescing, occupancy) and that this project is the CPU version of the same engineering questions.

17.8 Questions to ask them

- Which operations and data types (8-bit, 16-bit, float) dominate your library's use, and how do you decide when a hand-tuned path is worth maintaining?

- How do you validate numerical equivalence between a CPU reference and the optimized implementation (bit-exact, ULP tolerance, or application-level)?
- What does the performance-regression process look like: which benchmarks, which hardware, which noise tolerance?

Chapter 18

Drills and an eight-day plan

Reading is not enough for an interview: you have to produce explanations and small pieces of code on demand. These drills are ordered from easy to hard; do them on paper or at a whiteboard, *then* check against the guide.

18.1 Drills with short answers

1. **Bound check.** Show that $77R+150G+29B+128$ fits in 16 unsigned bits. *Answer:* $77+150+29=256$, so the maximum is $255\cdot 256+128=65\,408<65\,536$.

2. **Why 77, 150, 29?** *Answer:* $0.299\cdot 256=76.5$, $0.587\cdot 256=150.3$, $0.114\cdot 256=29.2$; rounded so the sum is exactly 256, so white stays 255.

3. **Sobel bound.** What are the largest possible $|G_x|$ and $|G_x|+|G_y|$, and which integer type do they need? *Answer:* 1020 and 2040; signed 16-bit.

4. **Mask.** Output byte i of the red channel for 16 pixels comes from byte $3i$ of the 48-byte block. Which input register and index does output byte 6 use? *Answer:* byte 18: register $\lfloor 18/16\rfloor = 1$, index $18 \bmod 16 = 2$.

5. **Throughput arithmetic.** A 4096×4096 gray image is converted in 5.0 ms. Pixels per second? Bytes per second counting 3 in and 1 out? *Answer:* $16.8\cdot 10^6/5.0\cdot 10^{-3}\approx 3.4\cdot 10^9$ px/s; $\times 4\approx 13.4$ GB/s. (Recompute with your own time; compare with your memcpy yardstick.)

6. **Working set.** Convolution $k=11$, $w=8192$: size of the ring? *Answer:* $4\cdot 11\cdot(8192+10)\approx 361$ KB: does not fit a 48 KB L1, fits a 1–2 MB L2 *(check the L2 size of your partition)*.

7. **FMA peak.** A core has two 256-bit FMA units at 2.5 GHz. Peak single-precision GFLOP/s? *Answer:* $2\cdot 8\cdot 2\cdot 2.5=80$.

8. **Arithmetic intensity.** I of a 5×5 convolution in flop per byte of minimum traffic? *Answer:* $2\cdot 25$ flop per 2 bytes $=25$ flop/byte.

9. **Amdahl.** 5 % serial, 16 threads: maximum speedup? *Answer:* $1/(0.05+0.95/16)=9.1$.

10. **Ties.** What is `rne(2.5)`, `rne(3.5)`, `rne(-0.5)`? *Answer:* 2, 4, -0 (then clamped to 0).

11. **FMA versus two roundings.** Why can `fma(a,b,c)` differ from `a*b+c`? *Answer:* the product is exact inside FMA before the single rounding of the sum; otherwise the product is rounded first.

12. **Speedup arithmetic.** Scalar 100 ms, AVX2 5 ms, 8 threads of AVX2 1 ms. Speedups and parallel efficiency? *Answer:* 20×, 100×; the thread scaling is $5/1=5\times$ on 8 threads, efficiency 62 %.

18.2 Code-reading drills

Open `src/avx2.cpp` and, without running anything, answer:

1. In the Sobel loop, why does the vector loop condition have to be `x + 16 <= w - 1`? What would break with `x + 16 <= w`, and which test would find it?
2. In the convolution, why is the ring indexed by *source row number*? What happens at the top border?
3. Which instruction fixes the lane order after `packus` in gray, which after `packs/packus` in the convolution, and why are they different?
4. What would change if the kernel weights were `double`?
5. Delete the `Kernel::validate` call. Name one input that now produces a wrong image without any error.

18.3 Whiteboard exercises

1. **Brighten.** Write an AVX2 loop that adds a constant to every byte of an image row with saturation (`_mm256_adds_epu8`), including the tail. State the tail strategy and the unaligned-load choice.
2. **Threshold.** 8-bit threshold: `out = in > t ? 255 : 0`. (Hint: there is no unsigned compare for bytes; use `max_epu8` and `cmpeq`, or bias by 128 and use the signed compare.)
3. **Horizontal sum of a vector.** Reduce 8 floats to one with the fewest instructions. State why a reduction is slower than elementwise work and why the convolution avoids it.
4. **Roofline sketch.** Draw the roofline of your sandbox or cluster core (peak flop/s and bandwidth from the memcpy yardstick) and place gray, Sobel and $k = 3, 9$ convolution on it. Say what changes if you use a second socket.
5. **GPU mapping.** Sketch the tile-with-halo shared-memory design for a $k \times k$ convolution. State the size of the shared tile for a 32×8 thread block and $k = 5$ producing one pixel per thread: $(32+4) \times (8+4)$ bytes.

18.4 Eight-day plan

Today is Wednesday 7 October 2026 and the exploratory interview is on Thursday 15 October at 11:00 CEST, so there are eight preparation days. Confirm the time slot and the interview format (video tool, who attends, whether a screen share or coding exercise is expected) by email now. If the other project guides share the same week, give this project the slots below and treat the cluster jobs as the long pole.

day	tasks
Wed 7	Unpack, `make test` locally if you have an x86 machine; read Chapters 1–3. Log in to the cluster and run `01_build_test.sbatch`. Submit the benchmark jobs as soon as the tests pass, because queue time is not under your control.
Thu 8	Chapters 4–6 and the gray chapter (7); explain the gray kernel aloud without notes. Check the first cluster results; fix anything that failed (the job logs show it).
Fri 9	Sobel and convolution chapters (8, 9). Drill 1–6. Submit repeat runs of the benchmark jobs for the repeatability comparison, and `-exclusive` for the scaling job.
Sat 10	Cache chapter (10), numerics (11). Generate your cluster figures with `make_figures.py -out`; fill in the bracketed numbers of Chapter 16.
Sun 11	Testing (12) and benchmarks (13); explain the mutation story and the ablation story aloud in two minutes each. Whiteboard exercises 1–3.
Mon 12	Interview Q&A (17), aloud, with a timer; record yourself once. Resume bullets: finalize against your measured numbers.
Tue 13	Whiteboard exercises 4–5 and the GPU-mapping questions; read the job description again and write three questions to ask. Mock interview with a friend or a recording: 30 minutes.
Wed 14	Light review only: one page of numbers (yours), the five soundbites, the limits (Chapter 13 caveats). Check audio, camera, network, and a plain note of the *allowed* materials.
Thu 15	Interview 11:00 CEST.

Interview rules

Use only what the interview invitation allows. Outside tools during the call, including AI assistants, are not permitted in your case and are grounds for disqualification; prepare so that you do not need them. Keep your notes to material you may use openly, and ask the recruiter in advance what is allowed if it is unclear.

Appendix A

Glossary

ablation An experiment that removes or changes one design decision to measure what it was worth.

accumulator A register that holds a running sum; the convolution keeps eight per block of 64 pixels.

Amdahl's law Speedup on n threads is at most $1/(s+(1-s)/n)$ if a fraction s is serial.

ASAN AddressSanitizer: a compiler option that checks every memory access for out-of-bounds use.

AVX2 Advanced Vector Extensions 2: 256-bit integer and floating-point vector instructions on x86-64.

band A contiguous group of image rows given to one OpenMP thread.

binomial kernel A smoothing kernel from Pascal's triangle; exact in binary floating point for small sizes.

broadcast Copying one scalar into every lane of a vector register.

cache line The 64-byte unit in which memory moves between cache levels.

clamp Limit a value to a range, e.g. to 0..255; also "clamp to edge", the border rule that repeats the outermost pixel.

correlation / convolution Sliding a kernel over an image and summing weight times pixel. Strictly, convolution flips the kernel; this library does the unflipped version (correlation), as most image libraries do.

deinterleave Separate packed RGBRGB… bytes into separate R, G, B vectors.

FMA Fused multiply-add: $a \cdot b + c$ with one rounding.

first touch On NUMA machines, a memory page is placed near the thread that first writes it.

GFLOP/s 10^9 floating-point operations per second; a multiply and an add count as two.

lane One element position inside a vector register (8 lanes of float in 256 bits).

Lmod / module The software-environment tool on the cluster.

memory-bound / compute-bound Limited by data movement versus by arithmetic throughput.

mutation testing Seeding bugs to check whether the tests notice.

NUMA Non-uniform memory access: memory attached to one socket is slower to reach from another.

OpenMP A compiler-directive standard for shared-memory parallelism (`#pragma omp …`).

pshufb Byte shuffle within each 128-bit half of an AVX2 register.

roofline A plot of attainable performance versus arithmetic intensity bounded by peak compute and memory bandwidth.

SIMD Single instruction, multiple data.

SIMT Single instruction, multiple threads: the GPU execution model.

Slurm The job scheduler of the cluster.

stride (pitch) The number of bytes from the start of one image row to the next.

tail The last few elements of a row that do not fill a whole vector.

ties-to-even Rounding rule: exact halves go to the nearest even integer.

TLB Translation lookaside buffer: the cache of virtual-to-physical address translations.

working set The data that a loop reuses and that should stay in cache.